

KairoX ERP — Production & Traceability

System, Business & Module Guide

Modules covered: Users · Employees · Clients · Imports · Barcode · Materials · Production · Drawers · Attendance · Wages · Analytics · Dashboard

Audience: this one document is written for three readers at once.

If you are...	Read	You will be able to
The client / business owner (non-technical)	Part A and Part B	Understand exactly what the software does on the factory floor, who touches it, and what happens to one garment from order to export
A frontend developer	Part A, Part B, Part C	Design and build every screen without needing live data or a backend engineer
A backend developer	Part B, Part D, Appendices	Review the code with full context — you will know what each file is for before you open it

Companion document: *KairoX Production-Event API Reference* — every endpoint with request/response examples and mock data.

Scope note. This document covers **Phase 1** — the system of record and the tracking layer, from the order sheet landing to the garment leaving the building. The Phase-2 procurement engine (BOM, procurement, inventory, supplier PO, intelligence) is documented separately in **KAIROX PROCUREMENT_SYSTEM_GUIDE** and **KAIROX PROCUREMENT_API_REFERENCE**.

Table of Contents

PART A — THE PLAIN-ENGLISH STORY

- 1. What this system is, in one page
- 2. The problem it solves
- 3. One garment's journey
- 4. Who does what
- 5. What "done" looks like at each step

PART B — THE BUSINESS FLOW IN DETAIL 6. Identity and access — who can log in at all 7. People — employees, cards, and the no-login rule 8. Clients, orders, styles and SKUs 9. Import and the release gate — where garments are born 10. The barcode system — the spine 11. Materials, lots and the style recipe 12. Production — one log, two doors, four gates 13. Drawers and the merge gate 14. Attendance 15. Wages 16. Analytics and the manager dashboards

PART C — FOR FRONTEND DEVELOPERS 17. The screen inventory 18. State machines you must render 19. Scan-driven UX — the patterns that matter 20. The error contract 21. Badges, chips and status vocabulary

PART D — FOR BACKEND DEVELOPERS 22. Architecture and the layering rules 23. Module map — every file and what it is for 24. The data model 25. Cross-module choreography 26. The key algorithms 27. Configuration and migration notes 28. A code-review checklist

APPENDICES A. Glossary B. Complete enum reference C. Audit log actions

PART A — THE PLAIN-ENGLISH STORY

1. What this system is, in one page

This is a production-management ERP for a **leather garment factory**. It manages the order-to-production lifecycle and gives **per-piece traceability**.

In one sentence: *every individual garment carries its own barcode from the moment its breakdown sheet is released, and every action anyone performs on it — cutting, stitching, storing, inspecting, exporting — is scanned and permanently logged against that barcode.*

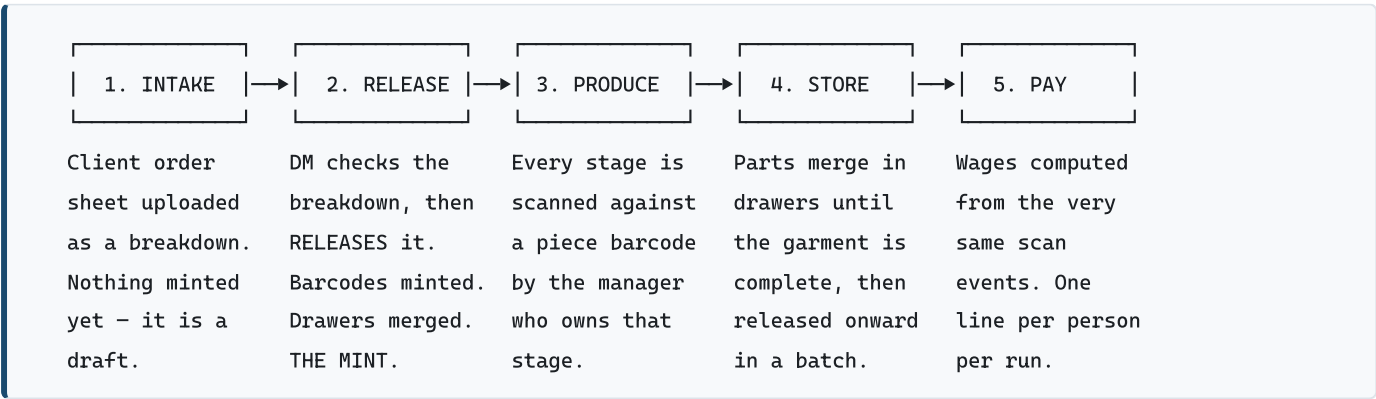
That single idea is what everything else hangs off.

What "per-piece" actually means

Most factory systems track **batches**: "200 jackets went to stitching today." This one tracks **pieces**: "garment PC-004821 — a size-50 black CLERMONT, the 17th of its SKU — was cut by Ramesh on the 4th from leather lot A-338, stored in drawer DRW-0042, line-stitched by Farid on the 9th, and is currently waiting for final inspection."

Every question a client, a manager or an auditor can ask about one garment has an exact answer, months later.

The five things the system does



Alongside those five, three supporting systems run continuously: **attendance** (who is on the floor today — production logging requires it), **materials** (what leather and accessories exist and what each garment consumed), and **dashboards** (what every manager needs to see about their own line).

2. The problem it solves

Before

1. An order sheet arrives. Someone types it into a spreadsheet.
2. Cutting happens. Someone writes "200 cut" on a whiteboard.
3. A bundle of parts moves to stitching in a plastic crate with a paper docket.
4. The docket gets wet, or swapped, or the crate gets split across two tables.
5. A client asks "which hide did the jacket in this complaint come from?" — nobody knows.
6. Payroll is worked out from tally marks in a notebook, two weeks late.
7. A stage runs out of work and nobody notices for a day.

After

Problem	How the system fixes it
Batches lose their identity when split	The piece is the tracked unit. There are no bundles. Each garment has its own barcode, so splitting a batch costs nothing
Someone logs the wrong stage	The caller never picks a stage. The login screen fixes the cut path; every other stage is inferred from the piece's own history
Work gets logged out of order	The sequence gate. A piece cannot be pasted before it is fused
A jacket reaches stitching with no lining	The merge gate. Line-stitching is blocked until the drawer holds everything that garment needs
The wrong worker is credited	The skill gate warns when a cutter is logged on a pasting job, and every event names the worker whose card was scanned
Someone logs work for a person who is not in the building	Production logging requires today's attendance
Leather consumption is guesswork	The two cut stages capture actual consumption per piece , decrementing a named lot
Payroll is a separate re-count	Wages are computed from the same production events . Nothing is counted twice
A whole batch is lost because one piece is wrong	Partial accept everywhere. One blocked piece never loses the thirty-nine good ones scanned with it
Workers need logins the factory cannot manage	Shop-floor workers have no login at all. They have a card. An operator scans it

The design principles behind all of it

Six rules run through every module. They explain most of the decisions in this document.

1. **The piece is the unit.** Not the bundle, not the batch, not the SKU.
2. **The scanner tells the server who and what — never what stage.** Stage is derived. A screen that lets someone pick a stage is a screen that eventually records the wrong one.
3. **Partial accept.** A batch operation reports per-item outcomes and commits the good ones. One bad scan never wastes a manager's whole tray.
4. **You retire codes, never people or records.** A worker leaving retires their card. Their employee row, every production event and every wage line stay intact for ever.
5. **Approval gates are hard, audited state transitions** — never a boolean someone can flip quietly.
6. **When the system is unsure, it asks.** Ambiguous material lots, undeclared lining, unconfirmed recipes — the system stops and surfaces the choice rather than guessing.

3. One garment's journey

Follow a single jacket — a size-50 black **CLERMONT** — end to end.

Step 1 — The order arrives

The client's order becomes a **ClientOrder** with a unique order number, holding **Styles** (CLERMONT), each holding **SKUs** (a colour + size combination with a quantity ordered).

```
Client — BOGGI MILANO
└─ ClientOrder — BOG-SS27-001
    └─ Style — CLERMONT (article A-4471, thickness 0.7mm)
        └─ SKU — BLACK · 50 · qty 20
            └─ Piece — seq 17 of 20 ← our jacket
```

Step 2 — The breakdown sheet is uploaded

The Direct Manager uploads the breakdown workbook. It is parsed and **previewed** first — nothing is written. When they commit, the styles and SKUs land as **DRAFT** rows.

Nothing is minted yet. No barcodes, no pieces, no drawers. The DM can now see the whole breakdown as an editable table and correct what the sheet got wrong — article numbers, thickness, colours, quantities, prices. Breakdown sheets are hand-made from a client order and a spec sheet, and the first pass is routinely wrong in small ways.

Step 3 — Release: the mint

When the breakdown is correct, the DM **releases** the styles. For each style they must answer one question that cannot be answered later:

Does this garment need a lining?

This is not optional and it is not inferred. `null` is rejected — "*we never asked*" is a real state and the system will not proceed from it. The answer is stamped on the style, copied onto every piece it mints, and from that moment decides two things: whether the garment has a lining-cutting stage at all, and whether its drawer must hold **both** parts before line-stitching.

Release then creates, atomically:

- the **Piece** rows — one per ordered unit, numbered 1..N within the SKU,
- each piece's **barcode** — a compact `PC-...` code that gets printed, plus a long human-readable alias,
- a **Drawer merge** — each piece is assigned a physical drawer from a finite pool.

Our jacket now exists. It is piece seq 17, barcode `PC-004821`, merged to drawer `DRW-0042`.

The drawer pool is finite — 200 drawers by default. If a release outruns it, the pieces are still minted and still have barcodes, but they have no drawer. They cannot be stored, so they cannot pass the merge gate. The response says exactly how many are waiting, and the DM can grow the pool.

Step 4 — Cutting

The **Cutting Manager** logs in. Their login is their screen — a cutting manager is pinned to the LEATHER_CUT screen and cannot be anywhere else.

At the table they scan: **the worker's card**, then **the piece barcodes** — a tray of thirty at a time. They choose the leather lot (or type article + colour and let the system resolve it) and enter the dcm consumed.

The server:

1. Checks the **role** may log leather cutting. ✓
2. Checks the worker's **designation** — a CUTTER, correct for this stage. ✓
3. Checks the **sequence** — cutting starts a chain, so nothing to check. ✓
4. Checks the worker is **present today**. ✓
5. Writes one ProductionEvent per piece, links the leather lot, and **decrements stock once for the whole batch**.

The lining is cut in parallel, by a different manager, on a different screen, against the same pieces.

Step 5 — Fusing and pasting

The **Stitching Manager** scans the same pieces at the next stages. They do not pick a stage — the server infers it. Piece **PC-004821** has completed LEATHER_CUTTING, so the next chain stage is FUSING. Scan it again tomorrow and the server infers PASTING.

If someone scans a piece for pasting that has not been fused, that one piece comes back **sequence_blocked** with the reason. The other twenty-nine in the tray still log.

Step 6 — The store

The cut parts go to the store. The scan order is **employee** → **drawer** → **piece**, and the drawer comes before the piece deliberately: the merge map is the authority, so a piece scanned into the wrong drawer is a **409**, not a re-assignment.

The system infers whether the part arriving is leather or lining from the piece's own cut history. Once the drawer holds everything that garment needs — leather, and lining if the style declared one — it **automatically reaches RECEIVED**. Nobody presses a button to assert a fact the server already computed.

Accessories are different. Scanning **part=ACCESSORY** issues the garment's whole accessory kit from the style's recipe and **spends stock**. It is never inferred — a wrongly guessed kit would move money nobody asked to move.

Step 7 — Send

The Store Manager opens the drawers list, filters to the **send queue**, ticks the drawers that are ready, and sends them as one batch. Sending sets each drawer to **SENDED** — and that is exactly what the production merge gate reads.

Our jacket's drawer is sent. **PC-004821** is now eligible for line-stitching.

Step 8 — Line stitching onward

The Stitching Manager scans it. This time the **merge gate** applies: is the drawer SENDED? Yes. The piece logs.

Then SHELL_STITCHING → FINAL_FINISH → FINAL_INSPECTION → PACKAGE_EXPORT. The last two are **approvals**, not floor work: no manager role owns them, so only DM/MD/HR can log them.

When PACKAGE_EXPORT is logged, drawer **DRW-0042** **recycles back to WAITING** and becomes available for the next garment.

Step 9 — Payroll

At the end of the fortnight, a **piece-rate run** is computed for the CLERMONT style. It reads the same ProductionEvent rows that were written on the floor, prices each against the rate in force **on the day the work happened**, and produces one line per employee.

The run is closed and **frozen**. It is the document the cash was counted against, and from that moment it is never silently recomputed.

Step 10 — Forever

Six months later a client complaint arrives about one jacket. Scanning its barcode returns: every stage, who worked it, on what date, whether it was reworked, which leather lot it was cut from, how much was consumed, which drawer held it, and what accessories it was issued.

4. Who does what

Role	Value	What they do
Managing Director	managing_director	Superuser. Bypasses every role gate. Final authority on payroll changes and deletions
Direct Manager	direct_manager	The operational lead. Uploads and releases breakdowns, manages clients and users, sets rates, runs payroll, grows the drawer pool. Also bypasses stage gates
Cutting Manager	cutting_manager	Logs leather cutting only . Pinned to the LEATHER_CUT screen. Creates material lots when leather arrives
Lining Manager	lining_manager	Logs the lining-cut path . Pinned to the LINING_CUT screen
Stitching Manager	stitching_manager	Logs every post-cut floor stage — fusing, pasting, line-stitching, shell-stitching, final finish. Also sends drawers
Store Manager	store_manager	Runs the store hub: scans parts into drawers, reads the drawers list, sends batches onward, records off-spec material issues. Store functions only — this role cannot log a production stage
Supervisor	supervisor	Reads the floor roster and dashboards. No attendance writes, no user creation
HR	hr	Employees, wages visibility, designation backfill, attendance operator
Security	security	Gate operator . Scans employee cards in and out. Nothing else
Merchandiser	merchandiser	Client-facing coordinator. Can log in; no route grants them anything yet
Client	client	External login, scoped to their own orders only
Viewer	viewer	Read-only office staff
Employee	employee	LEGACY — never minted . See below

The rule that shapes everything: workers get no login

A shop-floor worker has no login and no `app_user` row.

Creating one therefore needs no phone, no email and no password — just a name, a designation and a wage type. They are identified on the floor by their **employee barcode**, and their attendance is entered *for* them by an operator: **Security, HR, MD or DM**.

Why this matters:

- The factory does not have to manage hundreds of passwords for people who do not own smartphones.
- There is no "worker forgot their password" support burden.
- Attendance is done by a gate operator scanning cards, which is faster and more honest than self-service.

The `employee` role value still exists in the enum only because `app_user.role` is a native Postgres enum and values cannot be dropped — historical rows may still carry it. Nothing mints it any more.

Two separations worth understanding

The store manager cannot log production. The person who owns the store is not the person who owns the line. Store access and stage access are two different questions, and the code keeps them apart deliberately.

A cutting manager cannot log a lining cut. Each cut manager is pinned to their own screen by their role. The login is the screen. That is what makes "no stage buttons" possible.

5. What "done" looks like at each step

Step	Done when	Where you see it
Order exists	A ClientOrder row with a unique order number	<code>GET /imports/orders</code> — <code>breakdown_status: NOT_UPLOADED</code>
Breakdown committed	Styles and SKUs land as DRAFT	<code>breakdown_status: DRAFT</code> , <code>pieces_minted: 0</code>
Breakdown corrected	Article, thickness, colours, quantities all right	The breakdown table shows what you expect
Recipe confirmed	The style's material spec is signed off	<code>material_spec_confirmed_at</code> is set
Released	Pieces exist, barcodes minted, drawers merged	<code>production_status: RELEASED</code> , <code>minted_pieces > 0</code>
Drawers allocated	<code>pieces_waiting_for_drawer: 0</code>	The release response, and <code>GET /drawers/pool</code>
Cut	An event at LEATHER_CUTTING (and LINING_CUTTING if lined)	The piece's completed stages
Stored	The drawer holds everything the garment needs	Drawer state <code>received</code> , <code>can_send: true</code>
Sent	The drawer is SENDED	The piece is eligible for line-stitching
Finished	An event at PACKAGE_EXPORT	The drawer recycles to WAITING
Paid	A frozen wage run covering the work	Run status <code>closed</code>

PART B — THE BUSINESS FLOW IN DETAIL

6. Identity and access — who can log in at all

Module: `app/modules/users/` · **URL prefix:** `/api/v1/auth` and `/api/v1/users`

6.1 One table for every login

A single `app_user` table authenticates everyone — managers, HR, security, clients, viewers. Authorisation is then a matter of the `role` column.

Why one table and not one per role: authentication logic lives in exactly one place. Adding a role never means a new login flow or a new table — just a new enum value. And every "who did this" foreign key points at one column.

```
app_user
├─ phone          the LOGIN identifier (unique)
├─ email          optional, also unique
├─ password_hash  bcrypt
├─ role           the native PG enum
├─ is_active
├─ must_change_password
├─ employee_id    → this login IS a person on the payroll
└─ client_id      → this login is an external client, scoped to their orders
```

The two optional links are what let one table serve very different kinds of user without extra login tables. A manager who is also on the payroll has `employee_id` set. A client login has `client_id` set, and every read they perform is scoped to it.

6.2 The two gates

`require_roles(...)` — the per-route gate. MD and DM are **superusers** and bypass every one of them.

A note on that bypass: the spec target is for MD alone to be the superuser and DM to be operational-only. DM still bypasses during the MD rollout so the existing seeded god-account does not lose access. Drop DM from that set once an MD owner is confirmed in every environment.

`block_employees` — a deny-list dependency applied at the **router level** in `main.py`, so a *new* route is closed to the legacy employee role unless someone opens it deliberately. Most read routes use bare `get_current_user`; retrofitting an allow-list to every one of them is not realistic, and any route missed is a leak.

Three router groups stay **unlocked**: auth, users, and attendance — plus `/barcode/resolve`, which must stay reachable from the attendance screen so a gate operator can resolve a card.

6.3 Who may create whom

- `POST /users` admits DM, HR and MD at the door — but *which* role may be granted is decided in the service against the caller's own authority.
- `POST /users/clients` provisions a client login bound to a `client_id`. **DM and MD only — HR is deliberately excluded.** Binding a login to a client grants cross-tenant read access to that client's orders; that is a commercial decision, not an employee-admin one.

7. People — employees, cards, and the no-login rule

Module: `app/modules/employees/` · **URL prefix:** `/api/v1/employees`

7.1 Creating a worker

The common case is deliberately tiny:

```
{ "name": "Ramesh Kumar", "designation": "CUTTER", "wage_type": "piece_rate" }
```

No phone, no email, no password, no role. What comes back includes an **employee barcode** — minted in the same transaction so the card can be printed immediately.

7.2 Creating staff

Passing an explicit **staff** role mints a login alongside the employee row. That path *does* need a phone and a password.

Permitted here: **HR, Supervisor, Cutting/Lining/Stitching Manager, Security, Merchandiser**. Excluded: **DM and MD** (created via user-creation) and **employee** (workers get no login at all).

A rule that used to be wrong: a MONTHLY wage type used to auto-mint an **employee** login. It no longer does. Wage type is a payroll fact and says nothing about system access.

7.3 Designations — a controlled vocabulary

Designations are **always uppercase**, normalised on write: `" shell tailor "` and `"Shell-Tailor"` both become `SHELL_TAILOR`.

Why it is controlled and not free text: the moment it is free text, `Cutter`, `cutter` and `CUTTTER` become three different skills and the skill gate is decorative.

Unknown designations are **kept, not rejected** — seeded data carries job titles nobody has catalogued, and failing employee creation on an unrecognised title would block HR. The skill gate treats an uncatalogued designation as unknown and **passes** it (fail-open), tightening as HR backfills the vocabulary.

7.4 Names are unique

A collision is prefixed `IN-CHAL`. Two people called Ramesh Kumar are two rows, and the floor can tell them apart.

7.5 Leaving

Deleting an employee is a **soft delete**: `is_active: false`, and their barcode is retired. DM/MD only — **HR can edit but not remove**.

The employee row, every production event and every wage line stay intact. You delete the scannable code, never the person or their record. Scanning the retired card returns **410 Gone** — distinguishable from *"never existed"*, so the screen can say *"this card was deactivated"* rather than *"invalid barcode"*.

7.6 Salary is redacted

The roster returns salary **only** to HR, DM and MD. And the roster itself is internal — a CLIENT or VIEWER token is refused outright, because the row carries phone and email too.

8. Clients, orders, styles and SKUs

Module: `app/modules/clients/` · **URL prefix:** `/api/v1/clients`

8.1 The hierarchy

```
Client          name, country, code (KJ, GGZ...), currency, size system
├── ClientOrder  order_number (unique), dates, sea cut-off, ship mode
│   └── Style     name, article, thickness, code (unique), season, refs, price
│       └── SKU    colour + size + qty_ordered; code unique per style
│           └── Piece ONE physical garment; seq 1..N within the SKU
```

Piece is the tracked unit. Its `code` is the parent barcode.

8.2 Two dates that matter commercially

- `delivery_deadline` — when the client expects it.
- `sea_cutoff_date` — the last day to make sea freight.

Missing the sea cut-off means air freight, which is the single biggest margin event in the business. That is why it is an **alert** on every manager dashboard, not a report line.

8.3 Style-level production state

Each style carries its own release lifecycle:

Field	Meaning
<code>production_status</code>	<code>DRAFT</code> → <code>RELEASED</code> → <code>CANCELLED</code>
<code>released_at</code> / <code>released_by</code>	Who minted the pieces, and when
<code>needs_lining</code>	The DM's declaration. Nullable — null means unanswered
<code>material_spec_confirmed_at</code> / <code>_by</code>	The recipe sign-off
<code>material_spec_no_accessories</code>	The explicit " <i>this garment takes none</i> "

8.4 Tenancy

A CLIENT login is pinned to its own `client_id` on **every** client-facing read — the client list, the styles list, orders, analytics, production reads. A cross-tenant id resolves to 404 rather than 403, because **existence itself is information**.

8.5 Delete vs deactivate

`DELETE /clients/{id}` works only on a client with **no orders** — a mistyped or duplicate row. A client *with* orders is a **409**, because the cascade would take their orders, styles, SKUs, and every piece, production event and wage line hanging off them. The error names the exact call to make instead: `PATCH` with `{"is_active": false}`.

9. Import and the release gate — where garments are born

Module: `app/modules/imports/` · **URL prefix:** `/api/v1/imports`

This module is where a spreadsheet becomes real garments. It is the highest-consequence surface in Phase 1, and it is deliberately a **two-phase commit**.

9.1 Preview, then commit

```
POST /imports/preview — parse the workbook, return a summary + warnings. WRITES NOTHING.
POST /imports/commit — parse again, then write SKUs into the named order. Idempotent.
```

The order number is validated **first** — the endpoint 404s rather than create an order, so a typo cannot silently spawn a phantom order.

The upload itself is guarded three ways: extension, a hard **size cap** streamed rather than read into memory, and a **zip-container check** (an `.xlsx` is a zip; anything renamed to `.xlsx` that is not a valid zip is rejected before it reaches the parser).

9.2 Commit mints nothing

This is the change that matters most.

```
POST /imports/commit writes DRAFT styles and SKUs. It creates no pieces, no barcodes and no drawer merges. The response says so explicitly: release_required: true, pieces_minted: 0.
```

The sheet lands as an editable table. The DM reviews it and corrects what is wrong.

9.3 The breakdown table

`GET /imports/breakdown/{order_number}` is the screen between upload and production: styles with their SKUs, each row carrying its `production_status`.

- **DRAFT** rows are editable and have minted nothing.
- **RELEASED** rows have real barcoded garments behind them and are read-only.

Corrections are made through three doors:

- `PATCH /breakdown/skus/{id}` — the table row. **Can edit the SKU and its parent style in one call**, in one transaction, so the screen can never end up with the colour saved and the article not.
- `PATCH /breakdown/styles/{id}` — the style header, which has no SKU to hang off.
- `DELETE /breakdown/skus/{id}` — drop a line the sheet should not have had. DRAFT with zero minted pieces only.

All of them **409 once the style is RELEASED**: its pieces carry printed barcodes, and rewriting the breakdown behind a garment already on the floor is how a factory loses traceability.

9.4 The release — the mint

`POST /imports/breakdown/{order_number}/release` is the moment garments come into existence. Atomically, and only now:

1. **Piece** rows — one per ordered unit.
2. **Per-piece barcodes** — the compact `PC-XXXXXX` that gets printed, plus the long alias so older labels still resolve.
3. **The drawer merge** — each piece assigned a drawer from the pool.
4. The style stamped RELEASED with the actor and time, plus an `audit_log` row.

9.5 The lining declaration

Every style must declare whether it needs a lining. Three shapes are accepted:

Shape	Meaning
<code>{"styles": [{ "style_id": ..., "needs_lining": true }, ...]}</code>	The contract. Each style carries its own answer
<code>{"style_ids": [...], "needs_lining": true}</code>	Broadcast. One human answer covering the whole list — still a declaration
<code>{"style_ids": [...]} alone</code>	Deprecated. Falls back to <i>inferring</i> the lining requirement from the style name

A per-style answer **overrides** the broadcast, so the common case can be sent once and the exceptions named.

Three things make this worth the friction:

- **null is rejected.** "*We never asked*" is a real and different state from "*no*", and release will not proceed from it.
- **The declaration outranks the system's own guess, in both directions** — including declaring a style named "...KNIT" as leather-only. That is the point of asking: inference off a style name cannot know that a particular wool shell has no lining, and the DM holding the spec sheet can.
- **It cannot be changed after release.** Pieces are minted against it and their barcodes are printed.

The request body uses `extra="forbid"`, so a misspelt or misplaced field is a 422 naming it — rather than a silent no-op on the one field this endpoint exists to capture.

9.6 Partial accept

An already-released style comes back in `rejected` with its reason; the rest still release. Read `minted`, not the HTTP status.

9.7 The drawer-pool warning

Watch `minted.pieces_waiting_for_drawer`. The pool is finite. Pieces beyond the free drawers are minted with barcodes but nowhere to be stored — so they cannot pass the merge gate until a drawer frees up or the pool is grown.

`grow_drawer_pool: true` on the release call, or `POST /drawers/pool` afterwards, clears it.

9.8 The order index

`GET /imports/orders` is the index every breakdown screen opens from, with a rolled-up status per order:

<code>breakdown_status</code>	Meaning
<code>NOT_UPLOADED</code>	The order exists; no breakdown committed yet
<code>DRAFT</code>	Sheet uploaded, nothing released, nothing minted
<code>PARTIALLY_RELEASED</code>	Some styles in production, some still editable
<code>RELEASED</code>	Every live style released and minting pieces
<code>CANCELLED</code>	Every style on the order was cancelled

Ordered **most-recently-worked-on first** — the newest of the order's creation and its last-written style. The client's `order_date` says nothing about what the factory is handling now.

10. The barcode system — the spine

Module: `app/modules/barcode/` · **URL prefix:** `/api/v1/barcode`

10.1 One registry, one front door

`barcode_registry` is the single table every scan resolves through. Each printed code — a piece, a material lot, a drawer, an employee card — has exactly one row carrying its `type`, its `status`, and a nullable foreign key to the domain row it names.

`GET /barcode/resolve?code=...` is one indexed lookup:

Outcome	Meaning
404	Unknown code
410 Gone	Retired code — <i>"this was deactivated; the record and history are intact"</i>
200	The code's <code>type</code> plus a live payload for that type

That 404/410 split is the whole point. A retired employee card and a typo are different facts, and the screen should say different things.

Retirement applies to every barcode type, not just employee cards. A retired lot label and a deactivated piece must not silently resolve either.

10.2 The types

Type	Minted when	Editable?
PIECE	Release	No — permanent garment identity
LEATHER_LOT / LINING_LOT / ACCESSORY_LOT	Material lot created	No
DRAWER	Pool creation	No — static code, recycling state
EMPLOYEE	Employee created	Yes — reissue / deactivate

10.3 One piece, two codes

Since the compact code arrived, a piece has **two** active registry rows:

- the small `PC-004821` that is printed and scanned — the primary,
- its old long `KJ2451-CLERMONT-57-M-017` code, kept alive so labels printed before the change still resolve — flagged `is_alias: true`.

`is_alias` is a stored flag, not inferred from the prefix. Every per-order count and the whole history list select `type=PIECE` rows; with two rows per piece and no discriminator, `minted` doubles, `balance` goes negative, the duplicates integrity proof reads as broken, and the history table shows each garment twice. Those queries need to name the rule they mean — *"the code happens to start with PC-"* is not that rule.

A resolve on a legacy code still works and returns `is_alias: true`, so the screen can nudge the operator to reprint the label.

10.4 The employee card lifecycle — history is sacred

Reissue (lost or damaged card): retire the old code, mint a new one. **History untouched.**

Deactivate (worker leaves): flip the registry status to RETIRED → resolve returns 410. **The employee row, all production events, and all wage lines stay intact.**

Both are audited. `PATCH /employees/{id}/barcode` is where they live — mounted under `/employees` because that is where the frontend expects it, but owned by the barcode module because employee barcodes are a barcode concern.

10.5 Resolve answers the next question too

A scan does not just say *what* the code is. It also carries:

Field	Meaning
<code>next_expected_scan</code>	"PIECE" / "DRAWER" / null — which code to present next. Answered for every type, including EMPLOYEE, which is the first scan of every workflow
<code>next_stage</code> + <code>next_stage_label</code>	The production stage this piece is due at, derived by the same helper the write path uses
<code>next_stage_blocked_reason</code>	Set when that stage cannot be logged yet — e.g. the drawer has not been sent

That last pair matters: the stage is **still reported** even when blocked. The screen shows where the piece is going *and* what is holding it there.

Why one shared helper: `next_chain_stage()` is pure and takes a set of completed operation codes. Both the write path (which decides what a scan logs) and the read paths (which tell the screen what is coming) call it. Two copies of that loop would drift, and the drift is invisible until a screen offers a stage the log then refuses.

10.6 Printing

`POST /barcode/print` returns a Code128-ready payload for a set of codes, or for every piece of a SKU or order. Each label carries the `code` to encode plus a `label_line` — the order number, article, style, colour, size and serial, pre-joined and ready to typeset underneath.

There is also a **material barcode screen** (`GET /barcode/materials`), which exists because lots minted a code at creation and nobody could reprint it once the first label was lost.

10.7 The order-level integrity proof

`GET /barcode/orders/{id}/analytics` reports planned vs generated vs balance, per order and per style, plus:

- `duplicates` — should always be 0. It is the proof that minting stayed idempotent.
- `half_minted` — some pieces exist but not all.
- `fully_generated` — the order is complete.

11. Materials, lots and the style recipe

Module: `app/modules/materials/` · **URL prefixes:** `/api/v1/materials`, `/api/v1/suppliers`, `/api/v1/styles`

Two related things live here: **what the factory HAS** (lots), and **what a garment NEEDS** (the recipe). They meet at six identity columns and nowhere else.

11.1 Three categories, strict fields

Every lot must carry exactly its category's fields, or the API rejects it with a 422. Creating a lot **mints a child barcode** — that is how material formally enters inventory.

Category / subtype	Required attributes	Quantity field	UOM	Filters
Leather	thickness, dcm	dcm	dcm	article, colour, thickness
Lining / plain	thickness, mtrs	mtrs	mtrs	article, colour, thickness
Lining / ribs	kg	kg	kg	article, colour
Lining / knit	pcs	pcs	pcs	article, colour
Accessory / button	size, count	count	pcs	article, colour, size
Accessory / zip	size, count	count	pcs	article, colour, size
Accessory / thread	thickness, mtrs	mtrs	mtrs	article, colour, thickness
Accessory / other	description, count	count	pcs	article, colour

`article` and `colour` are required for every material. `GET /materials/spec?category=&subtype=` returns this list at runtime, so the frontend renders the right form and the right filter boxes without hard-coding anything.

A `(category, subtype)` pair not in that table is **rejected** — the system does not invent a shape for a material class it was not told about.

11.2 Three stored numbers

```

on_hand    a real column – physically in the building
reserved   a reservation ledger – committed to a required quantity
available  on_hand – reserved ← DERIVED, never stored

```

`available` is never stored, **so the two can never drift**.

Category-specific attributes live in a JSON column rather than twenty mostly-null columns. The four the queries actually filter on — article, colour, thickness, size — are promoted to real indexed columns.

11.3 The lot picker

`GET /materials/lots` is the endpoint the cut screen uses to turn "black sheep glass, 0.7mm" into a lot id.

Three material GETs, three different jobs — worth keeping straight:

Endpoint	Returns
<code>/materials/spec</code>	The form definition — which boxes to render
<code>/materials/lots</code>	The rows , with lot ids
<code>/materials/stock</code>	Aggregate totals , with the lot ids summed away

The picker also does two helpful things: `last_used_for_sku` pre-selects the lot this SKU was last cut from (and *shows* that it did), and passing `required` makes each lot report `covers_required` — so a lot that cannot cover the batch is greyed out **before** the cut instead of warning after it.

Exhausted lots are returned, not hidden. A manager searching for a lot they know exists must find it, with `available: 0` explaining itself.

11.4 Editing a lot — three different operations

The stock screen deliberately splits what looks like one action into three:

Operation	What it changes	Why separate
<code>PATCH /lots/{id}</code>	Identity — article, colour, thickness, size, supplier	
<code>PATCH /lots/{id}/adjust</code>	Quantity , as a <code>delta</code> with a mandatory reason, audited	It records a movement , so the ledger still adds up and someone can ask a year later why 40 dcm disappeared
<code>DELETE /lots/{id}</code>	Retires the lot and its barcode	Never a hard delete — cut events point at this lot and are the consumption history the costing screens are built on

Category, subtype, UOM and `on_hand` are not patchable. Moving LEATHER→LINING would leave a lot measured in dcm claiming to be metres, and every cut event already pointing at it would silently re-denominate. Setting `on_hand` directly would make the stock and the movement history disagree with no record of who changed it.

Retiring is presented as "*retire*", not "*delete*". A scan of the old label returns **410 Gone** ("*this lot was retired*"), not 404 ("*invalid barcode*").

11.5 Receiving

`POST /materials/receive` records **approved and rejected quantities separately**.

- Approved quantity is **added to stock**.
- Rejected quantity is **logged** — supplier quality history.
- A PO mismatch is a **409** unless a DM/MD sends `approve_mismatch: true`, which receives it into a **new substitute lot** rather than quietly re-labelling the old one.

11.6 Supplier orders

A simple two-state manual flow: **ORDERED** → **ARRIVED**. Raising an order validates the requested article against the chosen or suggested supplier's catalogue. `GET /materials/stock` suggests a supplier when short.

11.7 The style material spec — the recipe

Mounted at `/api/v1/styles/{style_id}/material-spec`. This is what one garment of a style needs: its leather, its lining, and every accessory.

```
StyleMaterialSpec (one row per line)
├─ style_id
├─ sku_id          null = the style-wide default; set = a per-SKU override
├─ category / subtype / article / colour / thickness / size
├─ qty_per_piece
├─ uom             DERIVED from (category, subtype) - never taken from the client
└─ material_lot_id optional explicit binding
```

The UOM is derived, not accepted. Buttons are pcs, thread is mtrs — the unit is a property of the material kind. `uom` stays in the request contract only so a client can round-trip a GET without stripping fields.

11.8 Confirming the recipe unlocks release

`POST /styles/{id}/material-spec/confirm` is the sign-off, and it is a **separate call from release** on purpose: the DM can finish the recipe days earlier, and the release screen can render `release_blockers` **before** the button is pressed rather than explaining a rejection afterwards.

`no_accessories: true` is the DM stating "*this garment takes none*". It is the only thing that lets a style with an empty accessory list through the release gate — **an empty list on its own is ambiguous** (nobody entered them yet?), and

releasing that silently is how a whole order reaches the store with no kit. Sending `true` while accessory lines exist is a 422, not a precedence rule.

11.9 The requirement screen

`GET /styles/{id}/material-spec/requirement` is `qty_ordered × per-piece` against what is on the shelf, per line.

This is the screen that should stop an order, and the moment to read it is before release. Afterwards the garments exist and a shortfall is a stoppage rather than a purchase order. `short_by` and `suggested_supplier` feed straight into `POST /suppliers/orders`.

11.10 The frozen recipe and its escape hatch

Once a style is RELEASED, its recipe **cannot be edited** — its garments are already being issued against it.

A wrong article is corrected by recording **what was actually handed over**: `POST /materials/issues` records a material issued to one garment outside its spec, and spends stock.

It is **repeatable on purpose**: three corrections on one garment are three real events, and none of them makes the kit checklist think a spec line was satisfied.

The store manager can call it — waiting for a DM to record a swapped button is how the correction stops being made at all.

12. Production — one log, two doors, four gates

Module: `app/modules/production/` · **URL prefix:** `/api/v1/production`

12.1 The pipeline

```
LEATHER_CUTTING ↴
                  └─ parallel cut paths, per piece
LINING_CUTTING  ↵
  → FUSING → PASTING → [ MERGE GATE ] → LINE_STITCHING → SHELL_STITCHING
  → FINAL_FINISH → FINAL_INSPECTION → PACKAGE_EXPORT
```

Two entry stages, one join. Leather and lining are cut independently, by different managers, on different screens. They have **no predecessor**. Everything after the drawer merge is a single linear chain.

That breaks the naive "one linear predecessor" model, so the code is explicit about it: `predecessor()` returns the leather-side predecessor for the leather chain, and **None for LINE_STITCHING** — because line-stitching's precondition is not "*pasting done*" but "*leather and lining both merged in the drawer*", which the **completeness gate** enforces, not the sequence gate.

Why stage order is in code, not config. The `operation` table stays the source of truth for labels and rates. But the stage **order**, the two parallel cut paths, the **role** → **stage** map and the **designation** → **stage** map are business law. Encoding order against `Operation.sequence` alone would mean a stray UPDATE to a sequence number silently reorders the factory. Codes are stable; sequence numbers are not.

12.2 One endpoint, two doors, no stage buttons

`POST /production/log` is the whole floor's logging surface. The caller sends an **actor** (an employee, by barcode or id) and **targets** (pieces, by barcode or sku + seqs).

The caller never sends a stage.

- A **cut screen** (LEATHER_CUT / LINING_CUT) fixes the cut stage.
- Otherwise the stage is **inferred** from each piece's own history — the next chain stage after the furthest one it has completed.

"Furthest completed, plus one" rather than "first not completed": a piece that was reworked or logged out of order should advance from how far it has actually got, not stall at the earliest gap.

The **screen context is derived from the role**. A cutting manager is pinned to LEATHER_CUT; a lining manager to LINING_CUT; a stitching manager or supervisor to PIPELINE. Any value they send is ignored. Only DM/MD/HR — who legitimately log any stage — may pass an explicit override.

Both doors POST the same shape. The router resolves barcodes to ids; the service sees ids only.

12.3 The four gates

Cheapest and most-likely-to-fail first.

#	Gate	Scope	On failure
1	ROLE — may this role log this stage?	Whole request	403
2	SKILL — may this designation work this stage?	Per piece	Warning, still logged
3	SEQUENCE — has the piece completed the previous chain stage?	Per piece	<code>sequence_blocked</code>
4	MERGE — for LINE_STITCHING only: is the drawer SENDED?	Per piece	<code>merge_blocked</code>

Gate 1 is whole-request because the role is wrong for the whole batch — there is nothing per-piece about it. **Gates 2–4 are per-piece** so that one bad piece never loses the thirty-nine good ones a manager scanned with it.

MD and DM bypass the role gate. HR does too.

Gate 2 is a warning, not a block. A cutter logged on a pasting job still records the work; the anomaly comes back in `skill_warnings` with the employee, their designation, the stage and a note. Blocking it would mean a helper covering a shift cannot be recorded at all.

Rework is permissive and bypasses gates 3 and 4. A piece returning to a stage it already passed is legitimate — flagged, never blocked.

12.4 The door gate

`POST /production/log` also carries a **role dependency at the door**, in front of the per-stage gate.

The route deliberately had none for a while, on the reasoning that the role gate is stage-specific. That is true for a piece with a resolvable stage — and it left a gap for one without. A piece that has not been cut yet resolves to *no* stage on the PIPELINE screen, so Gate 1 never runs and the request comes back 201 with everything in the `not_cut` bucket, for **any** authenticated token.

Nothing was ever written on that path, so it was not a data leak. It was worse as an **answer**: a store login was told *"nothing logged, cut it first"*, implying it may log once the piece is cut — when in fact its scan will 403 the moment a stage resolves. The role that cannot log anything is now told so at the door, once.

STORE_MANAGER is deliberately absent from the logger set.

12.5 Consumption

The **two cut stages only** capture material consumption per piece and **decrement stock once per batch**, in the same transaction as the events.

The lot link lives on the event — the act of cutting — never on the piece.

Three doors supply the lot:

1. **Lot id** — the screen already picked one.
2. **Article + colour + optional thickness** — the cutting manager types what is on the hide, and the router resolves it through the same lot picker the UI uses.
3. `use_style_spec: true` — take the dcm from the style's recipe.

Two of those refuse to guess:

- **No lot matches** → 404 naming the spec and telling you to add the delivery first.
- **Several lots match** → **409 naming up to five candidates** with their availability. Silently taking the first would decrement stock from a lot the manager never chose — a wrong number in the one ledger the factory reconciles against, and invisible.
- **A batch whose pieces disagree on the spec dcm** → 422 naming both values. Pieces from two SKUs with different overrides have no single number, and picking one would write the wrong consumption against half the garments.

`use_style_spec` is **off by default**, deliberately: omitting both `dcm` and the flag still returns today's 422, so no existing client silently changes behaviour on the one branch that guards the leather ledger. The recipe is normally reached the other way round — `/production/piece-state` returns `suggested_dcm_per_piece` and the screen prefills the field, so the operator still confirms the number that reaches the ledger.

A cut that consumes more than the lot has available is still recorded. Stock drifts and the garment is physically cut; the shortfall comes back in `stock_warning` rather than silently driving `on_hand` negative.

12.6 The scan-time read

`GET /production/piece-state` is what a barcode scan should return: **one piece**, not a SKU.

Send `code` + `employee_barcode` and read three fields:

Field	Meaning
<code>current_stage</code>	Where the piece is now
<code>next_stage</code>	What this scan would log — never chosen by hand
<code>ready_to_log</code>	<code>true</code> → POST the log immediately. <code>false</code> → show <code>blockers</code> . <code>null</code> → no employee was sent, so the question is unanswered — which is not the same as blocked, and must not be rendered as one

It exists because stage inference and all three per-piece gates used to run only at **write** time, so the UI had to guess: it made the operator pick a stage by hand, and it left later stage cards scannable before their predecessor was done. This answers both from the server, **using the very same predicates the log enforces** — so a card the UI opens is a card the log will accept.

It also carries the piece's `drawer`, its `stages[]` cards (each `completed` / `next` / `locked` / `not_applicable`, with the gate and reason for a locked one), the `actor` block (name, designation, whether they are checked in, any skill anomaly), the `material_requirement` kit checklist, and `sku` progress.

12.7 The response

`POST /production/log` returns a rich per-piece result, because a batch of forty is forty separate outcomes:

Field	Meaning
<code>stage</code>	The stage logged. "MIXED" when a PIPELINE batch spanned several; <code>null</code> only when nothing resolved
<code>stage_by_piece</code>	The truth for a mixed batch
<code>logged</code> / <code>rework</code> / <code>not_found</code> / <code>completed</code>	The classic buckets
<code>sequence_blocked</code> / <code>skill_blocked</code> / <code>merge_blocked</code> / <code>role_blocked</code>	Per-piece refusals
<code>blocked[]</code>	Every rejection with its cause — <code>{piece, stage, gate, reason}</code>
<code>skill_warnings[]</code>	Gate-2 anomalies that still logged
<code>stock_warning</code>	A cut that went short
<code>sku_progress</code>	<code>{total, done, remaining, closed}</code> for the stage just logged. <code>closed: true</code> means stop offering this style for scanning
<code>drawer_by_piece</code>	Each piece's drawer state
<code>kit_by_piece</code>	Deliberately lean — <code>kit_required</code> , <code>kit_status</code> , <code>outstanding</code> . A forty-piece batch carrying full requirement blocks would dwarf the rest of the response
<code>message</code>	One sentence for the scan screen

`preview: true` runs the whole thing as a dry run: computes every bucket, writes nothing.

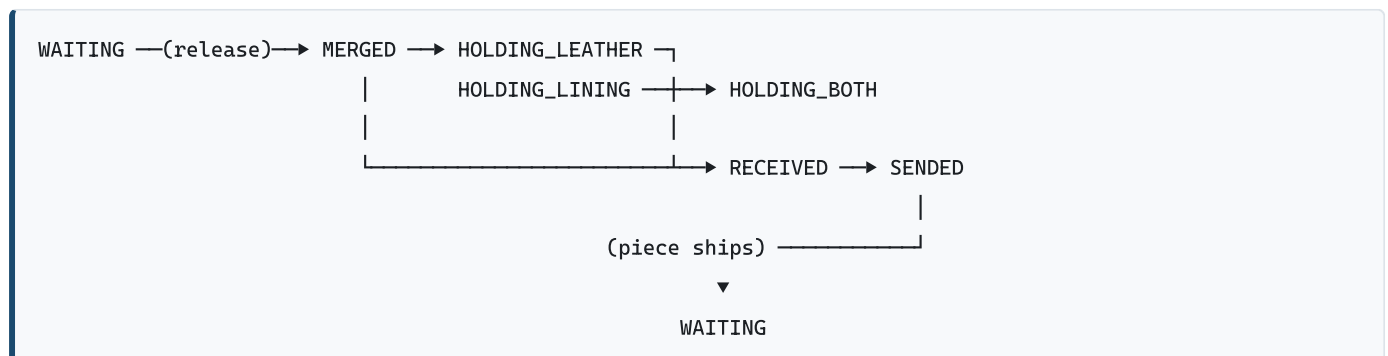
12.8 The retired routes

`POST /production/cutting` and `POST /production/scan` return **410 Gone** with a message pointing at `/log`. A deleted route 404s, and a 404 on a screen that worked yesterday reads as *"the server is broken"*. 410 says the removal was deliberate.

13. Drawers and the merge gate

Module: `app/modules/drawers/` · **URL prefix:** `/api/v1/drawers`

13.1 A static code with a recycling state



The **code never changes**. The state cycles. `current_piece_id` moves over the drawer's life.

13.2 The store scan

The order is **employee** → **drawer** → **piece**, and all three are enforced server-side.

- **The employee scan is mandatory.** The store scan used to accept no actor at all — the one door on the floor where "who did this" was unanswerable. A rule the frontend alone enforces is a rule that disappears the moment anything else calls the API.
- **The drawer comes before the piece** because the merge map is the authority. A piece scanned into the wrong drawer is a **409**, not a re-assignment.

`part` is **optional**. The server infers LEATHER vs LINING from the piece's own cut history and what the drawer is still missing. Send it only to override.

`part=ACCESSORY` is **explicit-only and never inferred**. The worst a wrong LEATHER/LINING guess can do is set the wrong boolean, which a human can undo. An inferred ACCESSORY would **spend stock** — it decrements every accessory lot on the style's spec — so a mis-inference would move money nothing on the floor asked to move.

The accessory scan is **idempotent**: a second tap issues nothing and returns the same 200.

Every scan returns the `kit` block, cut parts included, so the person at the drawer can see what accessories the garment still needs without leaving the screen.

13.3 Completeness, not sequence

The gate is about **completeness**, not order:

- A **lined** jacket needs leather **and** lining before its drawer is complete.
- A **leather-only** piece (`needs_lining: false`) is complete on leather alone.

RECEIVED is now reached automatically the moment a drawer holds everything its garment needs. The old flow — press RECEIVED, then press SEND, one drawer at a time — is gone, because RECEIVED asserted a fact the server had already computed.

`needs_lining` on every drawer row is the **EFFECTIVE requirement**, resolved from the style, the SKU and the cut history — **not** the stored `piece.needs_lining` flag, which is written once at release and is wrong for most of a live order. When the two disagree, `lining_reason` says why.

13.4 Send — the decision that stayed

SEND is a **decision**, so it stayed. And it became **plural**, because the store never releases one drawer at a time.

`POST /drawers/send` takes drawer ids and nothing else. **There is no destination to pick**: lining is upstream of the store, so a merged drawer has exactly one way forward.

Sending sets each drawer to SENDED, which is exactly what the production merge gate reads. The whole selected bunch of pieces becomes eligible for LINE_STITCHING in one action.

Partial accept: a drawer that is not yet RECEIVED comes back in `not_ready` with its reason; the rest are still sent. **Check `count_sent`, not the HTTP status.**

Who may send: MD, DM, Store Manager — **and the Stitching Manager**. That last one is a deliberate scope change: sending is what releases a batch into line-stitching, and on the floor the stitching manager is the one who walks to the store and takes the drawers. Requiring a DM to press the button made the store a bottleneck on a decision the stitching manager was already making physically. It is a **role grant on the store surface, not a production permission** — `STAGE_ROLE_ACCESS` is untouched. Cutting and lining managers are still excluded: they put parts **in**; they do not decide what leaves.

13.5 The drawers list

`GET /drawers` is the store's working screen.

Ordered most-recently-worked-on first by default. "Worked on" means the newest of: merged to a garment, part scanned in, received, sent, or released back to the pool. Every row carries `last_activity_at` and `last_activity` (`"merged"` / `"scanned"` / `"received"` / `"sent"` / `"released"`) — a timestamp with no verb tells the operator *when* something happened but not *what*, which is the half they need to decide whether to open the drawer.

This replaced an ordering that could not see three of those five events: a merge and a store scan wrote no timestamp at all, and a release nulled the received/sent stamps, so the drawer that had just shipped sank to the bottom.

Pass `sort=seq` for the print sheet — DRW-0001...DRW-0200 in drawer order.

Two search affordances work together: `code=` is a case-insensitive *contains*, so "42" and "drw-004" both work.

`pin_codes=` replays the client's own recently-searched list to float those drawers above everything else — nothing about a search is stored server-side, because that is one operator's session, not a fact about the factory. Pinned rows are flagged so the UI can band them separately.

`sendable=true` is the **send queue** — drawers that hold everything their garment needs, waiting for someone to tick and send them. `can_send` applies the same rule the send endpoint enforces, so **the queue can never offer a row the server refuses**.

A row with `barcode: null` is a drawer with no registry code. It cannot be scanned, so print nothing for it — and the list **shows** it rather than silently dropping it, because that is a broken label somebody needs to fix.

13.6 The pool

The pool is **fixed at 200** and grows only when a DM or MD says so.

`GET /drawers/pool` reports the pool size, free drawers, and how many minted pieces have **no drawer** — plus `shortfall`, what a DM would have to add right now to clear the list.

`POST /drawers/pool` adds N permanent barcoded drawers **and immediately drains the waiting list into them** — growing the pool while leaving pieces waiting would leave a DM staring at empty drawers beside a waiting list. It is capped at 1000 per call as a typo guard: `add: 20000` would mint twenty thousand permanent drawers and there is no un-mint.

`POST /drawers/allocate-waiting` merges drawer-less pieces into whatever drawers are currently free. Safe to call repeatedly; a no-op when there is nothing to place.

14. Attendance

Module: `app/modules/attendance/` · **URL prefix:** `/api/v1/attendance`

14.1 Every write comes from an operator

Only four roles may write attendance: SECURITY, HR, MD, DM.

Nobody else on the floor has a login, so there is nothing else it could be. Both doors share **one dependency**, so the barcode door and the manual door can never drift apart.

14.2 Two doors

The barcode door (primary): `POST /attendance/scan-check-in` with the card code and a direction of `"in"` or `"out"`. The operator standing at the gate scans every card — monthly or daily, including their colleagues'.

The manual door (fallback): `POST /attendance/proxy/check-in` and `/proxy/check-out` when a card fails or is forgotten. Takes a list of employee ids. Any wage type.

`/attendance/check-in` and `/check-out` are now an **operator recording their own** arrival and departure — their login is linked to an employee row. The body is optional; identity comes from the token and the timestamp is set server-side to block client-side clock manipulation.

14.3 One row per person per day

`work_date` uniqueness is enforced **at the database level**, so a duplicate check-in is physically impossible rather than merely discouraged. Check-in is **idempotent** — a re-tap is a no-op.

`recorded_by_user_id` names the operator on every proxy entry. Accountability, and a fraud trail.

Three flags are computed **at write time** so dashboards never recompute on read: `is_late`, `is_short`, `is_overtime`.

14.4 Shift policy is data, not code

A single `shift_config` row holds the start time, shift length, grace minutes and the factory timezone — so HR can change them without a redeploy. The timezone is the single source of truth for interpreting wall-clock policy and for deciding which calendar day a punch belongs to. Storage stays UTC; the timezone is applied only at the business-logic and display boundaries.

14.5 Location tracking is removed

No route asks for, validates or stores a position. Check-in succeeds on identity alone.

`lat` and `lon` are still **accepted and ignored** on the bodies that used to require them, so a client that still sends coordinates gets a 201 rather than a 422. The geofence columns remain mapped on the model because they are NOT NULL in Postgres with no server default — removing them for real is a migration, not a comment-out.

14.6 It gates production

Production logging requires the employee to be present today. That is the link between the two modules: work cannot be recorded for someone who is not in the building.

15. Wages

Module: `app/modules/wages/` · **URL prefix:** `/api/v1/wages`

15.1 Rates are date-effective

A rate is `(style, operation, rate, effective_from)`. A mid-period rate change prices **each day at the rate in force that day**. Nothing is retroactively re-priced.

The rate screens never show or accept a UUID — `style_code` and `operation_code` go in, and the service resolves them.

`GET /wages/styles` carries `rated_operations / total_operations` per style, so **unpriced styles are visible before payroll runs and silently pays zero for them**.

15.2 Two payrolls, not one

This is the central idea of the module. The fork is on the **employee population**, and it exists because the two populations are priced by incompatible facts.

run_kind	Pays	Scope	Why
piece	PIECE_RATE workers	style_code or order_number REQUIRED	A piece wage is earned on a specific garment, so the style is the run, not a filter on it
monthly	MONTHLY salaried staff	None required	A salary is a fact about a person for a period. There is no honest way to say what share of a fitter's month belongs to CLERMONT rather than CARNABY
combined	Both	—	Legacy only. Never created any more

The scope requirement on a piece run is the guardrail. Dates alone used to be accepted, and that is how a manager pays the whole factory's piece work under a window they meant to narrow. It is enforced at the schema (so it appears in the OpenAPI contract and the frontend can grey out the button before the round trip) **and repeated in the service** — because a rule only the HTTP layer knows is a rule that vanishes the moment anything else calls compute.

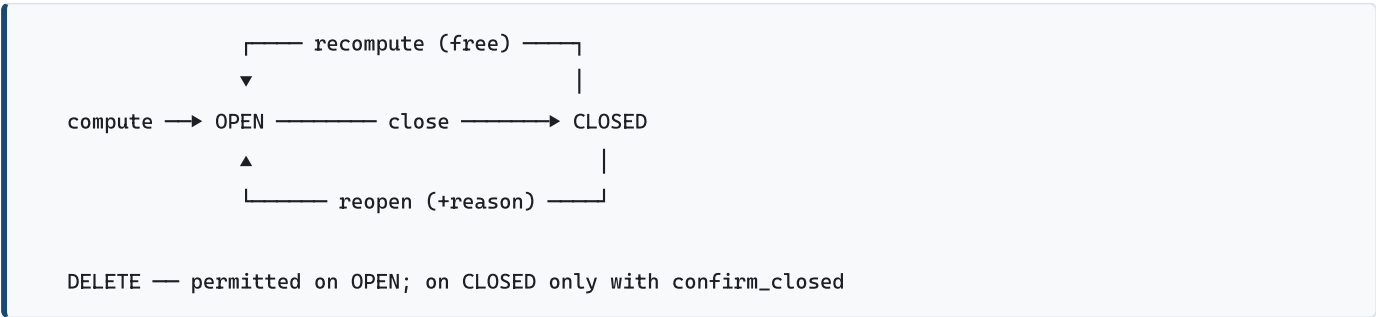
A monthly run **may** carry an order or style as a **label** for the payroll screen ("*the month we ran CLERMONT*"). The response echoes `scope_is_label: true` to say it did not narrow anything. **The screen must not print "CLERMONT payroll" over a sheet that paid every salaried person in the building.**

A piece run and a monthly run over the same window do not conflict. Their populations are disjoint, and together they are one complete payroll for that fortnight — compute both.

15.3 One line per employee per run

PIECE_RATE and MONTHLY are **mutually exclusive** — never both on one line. `pieces` is always 0 for monthly. Behind each summary line sits a `wage_line_detail` row per `(employee, style, operation)` cell, carrying the pieces, the rate applied and the amount. That is what makes the three breakdown folds possible.

15.4 The run lifecycle



Status	Meaning
OPEN	A draft . Lines exist and can be recomputed freely — nothing has been paid against them, so rewriting them costs nothing
CLOSED	Frozen . The document the cash was counted against. Never silently recomputed

Recompute discards the lines and rebuilds them from current production events and rates, for the same window and the same scope. The run keeps its id; `recompute_count` increments and the actor is stamped — so a payslip reprinted afterwards is **identifiably a different document** from the one paid against.

Reopen is the only door out of CLOSED, requires a **reason**, and is DM/MD only. It is deliberately a separate call from recompute: a manager who must press reopen, type why, then press recompute cannot rewrite a paid payslip by mistyping a run id. The reason is stored and belongs on every reissued payslip for that period.

`confirm_closed: true` on recompute is a one-call escape hatch. It works, it stamps the recompute, and it **records no reason** — which is exactly why the reopen door exists.

Close is idempotent (a double-click changed nothing) and **refuses to freeze a run with no lines** — locking a window in which nobody is paid is never what was meant.

Delete exists because the overlap guard has always named it. `create_run` commits an OPEN run **before** any line is written, so a compute that died halfway left a committed empty run permanently occupying that window, blocking every later run over those dates, with no way to clear it. It refuses a CLOSED run unless explicitly confirmed: deleting a frozen run destroys the record of a payment that actually happened. **To change a frozen run, reopen and recompute** — that keeps the audit trail. Deleting is for runs that should never have existed.

15.5 The overlap guard

Two runs that would pay the same money twice is a **409 naming the run that blocks you** and what to do about it.

Before paying anyone, check two fields on the response:

- `unrated_operations` — stages with no rate in force. They pay **zero**, silently, unless you look.
- `gap_days` — days in the window that no run covers.

15.6 The result screens

Endpoint	What it gives
<code>/runs/{id}/breakdown</code>	One frozen run folded three ways : per style (stages and workers nested), per stage factory-wide, and per employee
<code>/runs/{id}/pieces</code>	Per-piece detail — which garment, which stage, which worker, their employee barcode, and what that one piece paid
<code>/ledger</code>	Every computed run, latest computed first , searchable

All three breakdown folds come off the **same frozen row set**, so the three tabs cannot disagree with each other or with the run total — which they will if the frontend sums them itself.

Per-piece amounts come from the run's **frozen** rate for that cell. Nothing is re-priced, so a rate corrected after the run closed does not change what this says the run paid. A row with `amount: null` carries a `note` explaining why (a monthly worker, or the stage was unrated at run time) — **deliberately not a zero**, which would read as work worth nothing.

The ledger is ordered by **when a run was computed**, not by its period: the question the ledger answers is "*what did we just run*".

15.7 Who may see it

Wages are visible only to **HR, DM and MD**.

Every other read endpoint in this codebase uses bare `get_current_user`. These do not, because CLIENT is a real role with a real login, and **per-piece labour rates are the factory's cost structure**. A client who can see that cutting pays 12.50/pc on their own style is holding your margin during the next price negotiation.

HR **reads** payroll. DM and MD **authorise changes** to it — recompute, reopen, close and delete are all DM/MD.

16. Analytics and the manager dashboards

Modules: `app/modules/analytics/` and `app/modules/dashboard/` · **Prefixes:** `/api/v1/analytics`, `/api/v1/dashboard`

Both are **read-only** and **own no tables**. Every query reads other modules' models.

16.1 What moved, and why

The standalone **Analytics Overview** and **Risk Alerts** screens have been **removed** — they return **410 Gone** with a message naming the replacement.

The reasoning: the bottleneck and the alerts were the only parts of those pages anyone acted on, and **a manager should not have to know a separate screen exists to find out their line is blocked**. They moved onto the surface every manager already opens — `GET /dashboard/alerts` — readable by every manager role rather than the DM alone.

They are 410 rather than deleted for the same reason the old production routes are: a 404 on a screen that worked yesterday reads as *"the server is broken"*.

16.2 Analytics — the drill-down

```
/explorer → /orders/{id}/tree → /styles/{id}/detail → /pieces/detail
```

Plus two features that stand on their own:

- **The piece life story** (`/pieces/{code}/story`) — every stage a piece passed, who worked it, when, the rework flag, leather consumption at cutting, its current stage and what it is waiting on.
- **Consumption vs stock** (`/consumption`) — leather consumed per style, summed from cut events.

Analytics is a **live** view. Payroll of record stays the frozen wage run, and the two legitimately differ mid-period.

16.3 The five dashboards

Each is a **composite** — one call returns a whole screen.

Dashboard	For	Contains
Cutting	Cutting Manager	KPIs, current order, per-cutter performance, leather lots, per-order progress, 14-day trend
Lining	Lining Manager	The same shape for the lining leg, plus upcoming work — leather-cut pieces not yet lining-cut
Stitching	Stitching Manager	Top KPIs, the pre-store (fusing/pasting) and post-store (line/shell/final) blocks, the store handoff, the running style's funnel, per-stage employee performance
Store	Store Manager	Drawer KPIs, current styles in store, the filterable drawer grid, held drawers, empty drawers available for reuse
Direct Manager	DM / MD	The factory-wide control panel — composed from the four stage dashboards' own aggregates , so this screen and the four can never disagree

16.4 The alerts

`GET /dashboard/alerts` returns three blocks, most-actionable first:

Block	What it is
<code>bottleneck</code>	The deepest queue — the stage with the most work waiting in front of it. Deliberately <i>not</i> "the first unfinished stage": the first unfinished stage is wherever the line happens to have got to, which is not a constraint and not actionable
<code>stage_spread</code>	Where each downstream stage lags the leather cut. The gap is true WIP in flight
<code>freight_risk</code>	Orders approaching their sea cut-off. Missing it means air freight — the single biggest margin event in the business

`alert_count` is what a badge should render. **Empty lists are a good day, not a missing feature.**

16.5 Piece tracking — one handler, six URLs

A piece's history is the same history whichever screen is asking. `/dashboard/pieces/{code}` is canonical; five per-stage aliases exist so each dashboard keeps its own URL namespace.

Separate per-stage implementations would drift the moment one of them gained a field.

Three neighbouring reads answer genuinely different questions — worth keeping straight:

Endpoint	Answers
<code>/production/piece-state</code>	What happens next , and which stage cards to lock
<code>/dashboard/pieces/{code}</code>	The full stage history with the STORE overlay and consumption
<code>/analytics/pieces/{code}/story</code>	The analytics life story

16.6 Two honesty features

The consumption grid is one stage per response. It previously returned both cut stages while joining the leather lot, so lining-cut events appeared with a blank lot and their quantities counted toward leather totals. Asking for a consumption grid on any non-cut stage is a **422**, not a silently empty list — it is a question with no answer.

`meta.unsupported` on the **DM dashboard** names the figures that are null **by design** because no table backs them yet. Several quality and costing numbers fall into this. It is better to say so than to render a confident zero.

PART C — FOR FRONTEND DEVELOPERS

Everything here can be built against the mock payloads in the companion *API Reference*. You do not need a running backend.

17. The screen inventory

17.1 The single most important UI rule

The scan screen never asks the operator to pick a stage.

Send the piece barcode and the worker card to `GET /production/piece-state`, read `next_stage` and `ready_to_log`, and either post the log or render the blockers. An operator choosing a stage from a dropdown is an operator who will eventually choose the wrong one.

17.2 Import & release

Screen: Order index — `GET /imports/orders`. A searchable table with the rolled-up `breakdown_status` badge, `pieces_minted`, and a `breakdown_url` per row. Ordered most-recently-worked-on first.

Screen: Breakdown table

BOG-SS27-001 · BOGGI MILANO				DRAFT · 0 minted	
▼ CLERMONT	art A-4471	0.7mm	season SS27	[edit style]	
BLACK · 46 · 10		BLACK · 48 · 20	BLACK · 50 · 20		
COGNAC · 48 · 10		COGNAC · 50 · 10			
needs_lining: ⚠ NOT DECLARED					
▼ CARNABY	art A-4482	0.9mm	RELEASED · 40 ✓		
(read-only – pieces minted)					
[Release selected styles]					

Build:

- **DRAFT rows editable, RELEASED rows read-only.** Drive it off `production_status`; do not let the user discover the 409.
- Use the **combined PATCH** (`style: {...}` nested inside the SKU patch) so a row saves in one transaction. Two calls means a 200 on the SKU and a 409 on the style leaves the screen half-saved with no way to tell which half.
- Surface `needs_lining` per style **on this screen**, before the release dialog — a DM who spots it mid-review should be able to fix it there.

Screen: Release dialog — the highest-consequence dialog in the product.

```

Release 2 styles into production

This mints barcodes. It cannot be undone.

CLERMONT    60 pieces    Needs lining?    ( ) Yes    ( ) No    ▲
CARNABY     40 pieces    Needs lining?    (•) Yes    ( ) No

① Drawer pool: 143 free · 100 needed
[x] Grow the drawer pool if short

[ Cancel ]          [ Release ] ← disabled

```

Build:

- **Disable Release until every style has an explicit yes/no.** Never default the radio. `null` is a real state and the server rejects it.
- Offer a **"same answer for all"** control that maps to the top-level `needs_lining` — that is a declaration, not a guess.
- Show the drawer pool position **before** releasing. Read `GET /drawers/pool`.
- On the response, render `minted` per style and `pieces_waiting_for_drawer` **prominently** if non-zero.
- `rejected[]` entries are per-style with reasons. **Read `minted`, not the HTTP status.**

17.3 The scan screens

Screen: Cut screen (Cutting / Lining Manager)

```

LEATHER CUT                                Ramesh Kumar ✓ in

1. Scan worker card    ✓ Ramesh Kumar · CUTTER · present
2. Material            Article [SHEEP GLASS ▼] Colour [BLACK ▼]
                       Thickness [0.7 ▼] → lot A-338
                       Available 3,400 dcm ✓ covers this batch
3. dcm per piece       [ 34.5 ]    ① style recipe says 34.5
4. Scan pieces         ████████ 29 scanned

[ Log 29 pieces ]

```

Build:

- **The screen is fixed by the role.** Do not render a stage picker or a screen selector for a cutting or lining manager. Only DM/MD/HR get an override.
- Drive the material dropdowns off `GET /materials/lots` — `options` fills the cascading selects; the row with `last_used_for_sku: true` is pre-selected, **and you should show that you pre-selected it.**
- Pass `required` (dcm × piece count) so lots report `covers_required` and you can grey out a lot that cannot cover the batch **before** the cut.
- Prefill the dcm from `suggested_dcm_per_piece` on the piece-state read. **A suggestion, never a substitution** — the operator confirms the number that reaches the ledger.

- Handle the **409 with candidates**: several lots match the typed spec. Render the named candidates and let the operator pick.

Screen: Pipeline scan (Stitching Manager)

Scan piece ...
PC-004821 CLERMONT · BLACK · 50 · #017
Now: PASTING ✓ Next: LINE_STITCHING
Drawer DRW-0042 · holding both · SENDED ✓
✓ Ready to log [Log]

Blocked variant:

Now: PASTING ✓ Next: LINE_STITCHING
Drawer DRW-0042 · holding leather · waiting for LINING
🔒 Blocked merge - the drawer has not been sent Ask the store to complete and send this drawer.

Build:

- **ready_to_log** is the one boolean an automatic screen needs. **true** → post immediately. **false** → show **blockers**. **null** → **no employee was scanned, so the question is unanswered — do not render it as blocked**.
- Render **stages[]** as cards with four visual states: **completed**, **next**, **locked** (with its gate and reason), **not_applicable** (a lining cut on an unlined piece).
- Show the **actor** block — name, designation, checked-in status, any skill anomaly. **Skill is a warning, so it never appears in blockers**.
- **can_log_next: false** means this login's role cannot log the stage. Say "ask the stitching manager" rather than letting the scan come back 403.

Screen: Batch result — the most under-built screen in most implementations.

PASTING · 29 logged · 1 blocked
✓ 29 pieces logged ⚠ PC-004833 sequence - not yet fused ⚠ skill: Ramesh Kumar is a CUTTER, logged on PASTING (recorded) 🕒 SKU progress: 29 of 30 done · 1 remaining

Build:

- **Never present a partial batch as a failure**. 29 logged and 1 blocked is a good outcome.
- **blocked[]** carries **{piece, stage, gate, reason}** — render the reason, not just the code.
- **skill_warnings[]** are logged, not blocked. Style them differently from blockers.

- `sku_progress.closed: true` means stop offering this style for scanning.
- Handle `stage: "MIXED"` — read `stage_by_piece`.
- Offer `preview: true` as a "check before logging" affordance.

17.4 The store screens

Screen: Drawers list

Drawers [search: 42] ☒ send queue only sort: recent ▾					
DRW-	STATE	HOLDING	GARMENT	CAN SEND	ACTIVITY
0042	received	leather + lining ✓	#017	☒	received · 2m
0117	holding	leather only	#023	☐	scanned · 14m
0008	waiting	empty	—	☐	released · 1h
☒ 1 selected			[Send selected]		

Build:

- **state and holding are different questions.** State is the lifecycle; holding is what is physically inside. They stop agreeing at `received` and `sended`. Render both.
- Drive the Send checkbox off `can_send` — it applies the same rule the server enforces, so the queue can never offer a row the server refuses.
- Render `last_activity` with `last_activity_at`: `"sent · 2 min ago"`.
- Keep the client-side recently-searched list and replay it as `pin_codes`. Band pinned rows separately — a pinned drawer from last week must not look like the busiest drawer on the floor.
- A row with `barcode: null` is a **broken label**: show it, do not print it.
- Use `needs_lining` from the row (the effective requirement), and show `lining_reason` when present.

Screen: Store scan — enforce the order visually: employee → drawer → piece. All three are enforced server-side anyway, but the screen should make the sequence obvious.

Render the `kit` block on **every** scan, not just accessory ones — the person holding the leather is the one who also has to find the buttons. `status: NOT_REQUIRED` means **hide the checklist**, not render an empty one. "No accessories declared" and "accessories declared but none issued" are different facts.

On an accessory scan, show `issued_now`, `already_issued`, `outstanding` and — importantly — `unresolved`, where a line could not be matched to a lot (`reason: NONE` or `AMBIGUOUS` with candidate lot ids).

`auto_received: true` deserves a visible confirmation: *"This drawer is now complete and ready to send."*

`sent: false` plus `next_action` exists because scanning is **not** completion. Say so explicitly.

17.5 Attendance

The gate screen is one input and two buttons: scan a card, in or out. Show the resolved worker's name and photo-less identity card immediately, then the result — late flag, whether this was a re-tap (idempotent no-op).

Keep the manual door one click away for a failed card. `/attendance/today` is the roster.

17.6 Wages

Screen: Order cards → **style cards** → **rate sheet**. Three levels. The `n of m priced` badge at each level is the point — unpriced styles pay zero silently.

Screen: Compute run — this dialog decides which of two incompatible pricing rules applies.

Compute payroll

Kind (•) Piece rate () Monthly salary

Scope Style [JP-CLERMONT_VEST ▾] ← REQUIRED for piece

Window [2026-09-01] → [2026-09-15]

☒ Freeze on compute

ⓘ A piece run and a monthly run over the same window do not conflict. Compute both for a full payroll.

- **Grey out Compute for a piece run with no scope.** The schema rejects it; do not make the user find out.
- For a monthly run, if they set a scope, label it *"for reference only"* and echo `scope_is_label: true` on the result.
- On the response, **surface `unrated_operations` and `gap_days` before anyone approves a payment.**
- The 409 overlap error **names the blocking run** — link to it.

Screen: Run detail — three tabs from `/breakdown` (style / stage / employee) plus a per-piece tab. All four come off the same frozen rows; **never sum them client-side.**

`scope_is_label: true` must render differently from a genuine narrowing. `recompute_count > 0` or `reopen_count > 0` should be visible on the header — a rebuilt run is a different document from the one paid against.

17.7 Dashboards

Each is one call. Render the composite. Read `meta.unsupported` and render those figures as *"not tracked"* rather than as zero.

The alerts block belongs on **every** manager dashboard, with `alert_count` as a badge.

18. State machines you must render

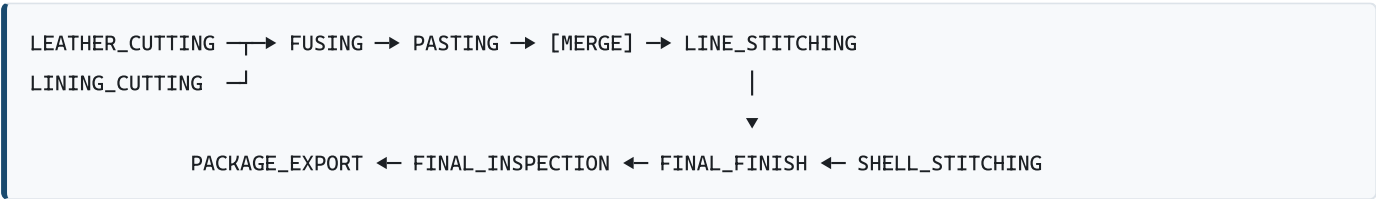
Style production status

From	Action	To	Guard
DRAFT	release	RELEASED	Every style must declare <code>needs_lining</code> . Mints pieces + barcodes + drawers
DRAFT	cancel	CANCELLED	
RELEASED	(nothing)	—	Frozen. Re-release only tops up quantity

Drawer state

From	Trigger	To
waiting	release merges a piece	merged
merged	leather scanned	holding_leather
merged	lining scanned	holding_lining
holding_leather	lining scanned	holding_both
any holding	completeness reached	received (automatic)
received	send	sended
sended	PACKAGE_EXPORT logged	waiting

Production stage chain — forward-only, except rework which is always permitted.



Wage run

From	Action	To	Guard
—	compute	open closed	freeze decides. Piece runs need a scope
open	recompute	open	Free
open	close	closed	Refuses an empty run. Idempotent
closed	reopen	open	DM/MD + a reason
closed	recompute	closed	Only with confirm_closed — no reason recorded
open	delete	—	
closed	delete	—	Only with confirm_closed

Barcode status — active → retired . One way. Resolve returns 410 after.

19. Scan-driven UX — the patterns that matter

19.1 The resolve-first pattern

Every workflow starts with a scan of unknown type. Call GET /barcode/resolve?code=... and branch on type :

```
const r = await resolve(code);
switch (r.type) {
  case "EMPLOYEE":      return setActor(r.employee);           // next_expected_scan tells you what's next
  case "PIECE":         return setPiece(r.piece);
  case "DRAWER":       return setDrawer(r.drawer);
  case "LEATHER_LOT":
  case "LINING_LOT":
  case "ACCESSORY_LOT": return setLot(r.lot);
}
```

Handle the three outcomes distinctly:

Status	Say
404	"Unknown barcode."
410	" <i>This card was deactivated.</i> " — never "invalid"
200 + <code>is_alias: true</code>	Works, but " <i>this is an old label — reprint it.</i> "

Use `next_expected_scan` to drive the wizard. It is answered for every type, including EMPLOYEE — the first scan of every workflow.

19.2 The state-then-log pattern

```
scan piece + scan worker
  |
  ▼
GET /production/piece-state?code=...&employee_barcode=...
  |
  ├── ready_to_log: true   → POST /production/log immediately
  ├── ready_to_log: false → render blockers[], offer no Log button
  └── ready_to_log: null   → "scan the worker's card" (NOT "blocked")
```

The read uses the same predicates as the write, so **a card the UI opens is a card the log will accept.**

19.3 Batch accumulation

Cut and pipeline screens accumulate scans, then post once. Show a running count, allow removal, and offer `preview: true` before committing a large tray.

19.4 Never poll

Nothing in Phase 1 is asynchronous. Every write is synchronous and returns its full result. There is no 202, no task id, no SSE. Post and render.

(The Phase-2 procurement engine does use 202-and-poll and SSE. Do not carry those patterns over.)

20. The error contract

20.1 The shape

FastAPI's `detail` wrapper, containing either a plain string or a structured object:

```
{ "detail": "No drawer with code 'DRW-9999'." }
```

```
{ "detail": { "error": "stale_revision", "current_revision": 7 } }
```

Handle both.

```
function errorMessage(body) {
  const d = body?.detail;
  if (typeof d === "string") return d;
  if (Array.isArray(d)) return d.map(e => `${e.loc?.join(".")}: ${e.msg}`).join("; ");
  return d?.message ?? d?.error ?? "Something went wrong.";
}
```

Validation failures from Pydantic come back as an **array** of `{loc, msg, type}`. Several of this codebase's most useful 422s are Pydantic messages written as full sentences — render them verbatim.

Every `500` carries a `request_id`. **Show it** — support traces the failure by it.

20.2 Status codes

Code	Means	Do
200 / 201	OK	
204	No content	Password change, client delete
400	Bad upload	Not an <code>.xlsx</code> , or not a valid zip
401	Unauthenticated	Redirect to login
403	Wrong role	The message usually names who can
404	Not found	Also: unknown barcode
409	State conflict	See below
410	Gone	A retired barcode, or a removed endpoint
413	File too large	Show the cap
422	Validation	Field-level, or a domain sentence
500	Server error	Show <code>request_id</code>

20.3 The 409s and 410s worth handling individually

Situation	Meaning	UI response
Released style edited	Frozen — barcodes printed	Drive editability off <code>production_status</code>
Piece scanned into wrong drawer	The merge map is the authority	" <i>This garment belongs in DRW-xxxx</i> "
Ambiguous material lot	Several lots match the typed spec	Render the named candidates and let them pick
Lot retire with stock reserved	Reservations outstanding	Say what is holding it
Lot spec collision	Two lots a cutter cannot tell apart	Offer the existing lot
Wage run overlap	Would pay the same money twice	Link to the blocking run
Recompute a closed run	Frozen	Offer Reopen (with a reason field)
Delete a closed run	Destroys a payment record	Require explicit confirmation
Client delete with orders	Cascade would take everything	Offer deactivate instead
410 on a barcode	Retired card / lot / piece	" <i>This was deactivated</i> "
410 on an endpoint	Removed deliberately	The message names the replacement

20.4 The 422s that are really business rules

Where	Message
Release	" <i>N styles have no lining answer... it cannot be changed after release</i> "
Piece wage run	" <i>A piece-rate run must name the work it pays for</i> "
Spec confirm	<code>no_accessories: true</code> while accessory lines exist
Consumption	" <i>The pieces in this batch have different consumption in their style specs</i> "
Employee create	" <i>password is only accepted when a staff login is created</i> "
Consumption grid	" <i>stage must be one of [LEATHER_CUTTING, LINING_CUTTING]</i> "

Render these verbatim. They are written to be read by the operator, and they explain the rule as well as the failure.

21. Badges, chips and status vocabulary

Family	Value	Colour	Note
Style production	DRAFT	grey	Editable
	RELEASED	green	Frozen; barcodes exist
	CANCELLED	red, struck	
Breakdown rollup	NOT_UPLOADED grey · DRAFT amber · PARTIALLY_RELEASED blue · RELEASED green · CANCELLED red		
Drawer state	waiting grey · merged blue · holding_leather / holding_lining amber · holding_both teal · received green · sende purple		Purple: it has left the store
Drawer holding	EMPTY · HOLDING LEATHER · HOLDING LINING · HOLDING BOTH		Different question from state
Stage card	completed green ✓ · next blue, prominent · locked grey 🔒 · not_applicable faint		
Log outcome	logged green · rework amber ⤴ · not_found grey · completed teal		
	sequence_blocked / merge_blocked / role_blocked red		
	skill_warnings amber — logged anyway		Never red
Barcode	active green · retired grey struck · is_alias amber "reprint"		
Kit	NOT_REQUIRED hidden · PENDING grey · PARTIAL amber · ISSUED green		Hidden, not empty
Attendance	present green · is_late amber · is_short amber · is_overtime blue · absent grey		
Wage run	open amber "draft" · closed green 🔒 "frozen"		
	scope_is_label: true → "label only" chip		
	recompute_count > 0 → "rebuilt" chip		
Stock	available > required green · 0 < available < required amber · available = 0 red		
Freight	past sea cut-off red · within 7 days amber · else green		The biggest margin event

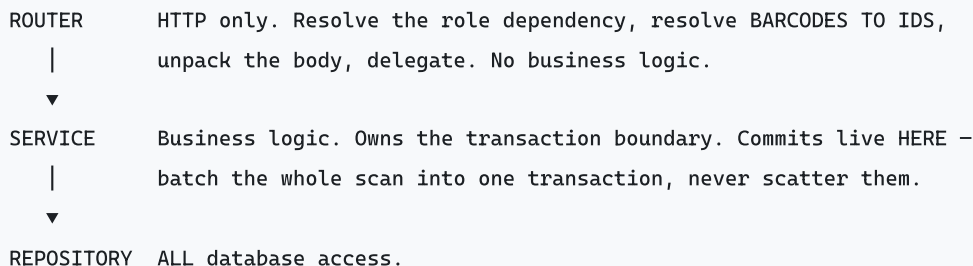
PART D — FOR BACKEND DEVELOPERS

22. Architecture and the layering rules

22.1 A modular monolith

One deployable FastAPI app, twelve internal modules (plus five Phase-2 ones). Cross-module calls go **through services**, never repositories or models.

22.2 The three layers



Barcode resolution happens in the router. That is deliberate and load-bearing: the service then sees ids only, so the barcode door and the manual door converge before any business logic runs and cannot behave differently.

22.3 Where business law lives

Four things are **code, not config**, and `core/enums_barcode.py` is where they live:

1. **Stage order.**
2. **The two parallel cut paths.**
3. **Role → stage access.**
4. **Designation → stage access.**

The `operation` table remains the source of truth for **labels and rates** — the MD edits those. But encoding order against `Operation.sequence` would mean a stray UPDATE silently reorders the factory. Codes are stable; sequence numbers are not.

`operation_access` is a runtime **escape hatch** on top of the enum default, so an MD can grant an exception without a deploy.

22.4 One helper, both paths

`next_chain_stage(completed_codes)` is pure — a set of codes in, a stage out, no database.

It exists so the **write** path (which decides what a scan logs) and the **read** paths (`/barcode/resolve` , `/production/piece-state`) cannot answer the same question differently. Two copies of that loop would drift, and the drift is invisible until a screen offers a stage the log then refuses.

When you add a stage, add it here and both sides move together.

22.5 Async discipline

Every blocking call goes through `run_in_threadpool` — openpyxl parsing, the sync bulk loader, drawer-pool operations. The event loop is never blocked.

The import path deliberately keeps the **proven sync loader** and runs it in a worker thread rather than rewriting it as `async`.

23. Module map — every file and what it is for

23.1 `users/` — 502 lines

File	Lines	Purpose
<code>service.py</code>	200	Login, user creation with authority checks, password change
<code>deps.py</code>	108	<code>get_current_user</code> , <code>require_roles</code> , <code>block_employees</code> — the whole auth surface
<code>router.py</code>	97	6 endpoints across two routers
<code>schemas.py</code>	68	Token, UserRead, the two create shapes
<code>repository.py</code>	66	
<code>models.py</code>	63	<code>app_user</code> — the one login table

Why `deps.py` is in `users` and not `core`: it depends on the concrete `User` model, and core must not import modules (enforced by `import-linter`). `core/security.py` keeps only the model-agnostic primitives.

23.2 `employees/` — 642 lines

File	Lines	Purpose
<code>service.py</code>	279	Create (with optional staff login), update, soft-delete, barcode lookup
<code>schemas.py</code>	134	The login-role validation rules live here
<code>router.py</code>	97	4 endpoints
<code>repository.py</code>	80	
<code>models.py</code>	52	

23.3 `clients/` — 1,116 lines

File	Lines	Purpose
<code>repository.py</code>	360	
<code>models.py</code>	310	Client → Order → Style → SKU + StyleComponent + SkuOrderLine
<code>service.py</code>	228	Including <code>create_order_with_breakdown</code> (called by Phase 2)
<code>router.py</code>	173	8 endpoints. Route order is load-bearing
<code>schemas.py</code>	136	
<code>utlis.py</code>	85	(sic)

23.4 imports/ — 3,234 lines

File	Lines	Purpose
<code>breakdown.py</code>	834	The breakdown table, edits, cancel, and RELEASE
<code>premint.py</code>	579	The mint — pieces, barcodes, drawer merges, pool management
<code>router.py</code>	536	9 endpoints. Also holds the release-request validation
<code>load_to_db.py</code>	423	The idempotent bulk loader
<code>_size_band.py</code>	398	Size-band parsing
<code>parse_orders.py</code>	299	
<code>excel_reader.py</code>	186	
<code>import_engine.py</code>	168	<code>build_preview</code>
<code>parse_production.py</code>	160	

23.5 barcode/ — 1,957 lines

File	Lines	Purpose
<code>repository.py</code>	724	Every read. No SQL lives in the service
<code>service.py</code>	642	<code>resolve</code> , print payloads, employee-card lifecycle, order analytics
<code>models.py</code>	468	<code>barcode_registry</code> , <code>drawer</code> , <code>material_lot</code> , reservations, receipts, suppliers, supplier orders, style spec, issue ledger
<code>router.py</code>	178	10 endpoints
<code>schemas.py</code>	145	

23.6 materials/ — 2,855 lines

File	Lines	Purpose
<code>style_spec_service.py</code>	1097	The recipe — CRUD, confirm, copy, requirement, manual issue
<code>service.py</code>	832	Lots, stock, receiving, supplier orders, the decrement
<code>router.py</code>	424	Three routers — <code>/materials</code> , <code>/suppliers</code> , <code>/styles</code>
<code>repository.py</code>	212	
<code>style_spec_repository.py</code>	199	
<code>schemas.py</code>	200	
<code>style_spec_schemas.py</code>	91	

23.7 production/ — 2,287 lines

File	Lines	Purpose
service.py	1109	log_batch and the four gates; piece_state
router.py	377	9 endpoints. Resolves barcodes and lots to ids
repository.py	336	
schemas.py	327	The rich LogResult and PieceState
models.py	130	operation, operation_access, style_operation, piece, production_event

23.8 drawers/ — 1,844 lines

File	Lines	Purpose
service.py	1193	Store scan, part inference, completeness, auto-receive, batch send, the kit issue
router.py	406	9 endpoints
schemas.py	245	

No models.py — the drawer table lives in barcode/models.py.

23.9 attendance/ — 1,063 lines

File	Lines	Purpose
service.py	471	_open_or_reject / _close primitives, shared by both doors
router.py	209	12 endpoints
schemas.py	164	
models.py	101	shift_config singleton, attendance_log
repository.py	61	
geofence.py	44	Vestigial — the feature is removed

23.10 wages/ — 3,074 lines

File	Lines	Purpose
service.py	1241	Rates, the two run kinds, the overlap guard, recompute/reopen/close/delete, the folds
repository.py	742	
schemas.py	423	RunRequest carries the scope rules in the contract
router.py	387	16 endpoints. Route order is load-bearing
models.py	175	rate, wage_run, wage_line, wage_line_detail
proration.py	68	Monthly calendar pricing

23.11 analytics/ — 1,365 lines · dashboard/ — 4,425 lines

File	Lines	Purpose
analytics/service.py	1056	Explorer, drill-downs, life story, consumption, alerts
analytics/router.py	153	10 endpoints — 3 of them 410 Gone
analytics/barcode_ext.py	149	
dashboard/repository.py	1944	Every grouped query
dashboard/service.py	1209	Five composites
dashboard/schemas.py	760	The typed dashboard shapes
dashboard/router.py	493	22 endpoints

Neither module owns a table.

24. The data model

24.1 Conventions

`UUIDMixin` (portable `GUID`) + `TimestampMixin` on every table. Money `Numeric(12,2)` ; quantities `Numeric(12,3)` or `(14,3)` ; semi-structured data `JSON_VARIANT` .

Status columns are `VARCHAR` storing a `str` -Enum's `.value` — with one exception.

24.2 The one native enum

`app_user.role` is a native Postgres enum (`user_role`). Everything else in Phase 1 is `VARCHAR`.

That has a sharp migration consequence — see §27.2.

24.3 The tables

IDENTITY

app_user one login table for everyone

PEOPLE

employee name, designation, wage_type, salary, is_active

COMMERCIAL

client name, country, code, currency, size system

client_order order_number, dates, sea_cutoff, ship_mode

style + production_status, needs_lining, spec confirmation

style_component VEST, FUR DETACH, KNIT, NYLON add-ons

sku colour + size + qty_ordered

sku_order_line per-date order lines under a SKU

PRODUCTION

operation code, label, sequence (labels + rates = config)

operation_access role → operation runtime override

style_operation which operations a style actually uses

piece ★ ONE GARMENT. (sku_id, seq) unique. needs_lining, drawer_id

production_event ★ THE CENTRAL TABLE. one worker, one op, ONE PIECE, one day
+ leather_lot_id / lining_lot_id / consumption_qty at cuts

BARCODE & STORE (all in barcode/models.py)

barcode_registry ★ every scannable code. type, status, is_alias, 7 nullable FKs

drawer static code, recycling state, current_piece_id

material_lot category/subtype/article/colour/thickness/size, on_hand, attributes

material_reservation soft allocation

material_receipt approved / rejected quantities

material_supplier the vendor directory

supplier_order ORDERED → ARRIVED

style_material_spec ★ THE RECIPE. style or SKU-scoped lines

piece_material_issue ★ THE ISSUE LEDGER. what was actually given to one garment

ATTENDANCE

shift_config singleton policy row

attendance_log one row per (employee, work_date) – DB-enforced

WAGES

rate (style, operation, rate, effective_from)

wage_run window + status + run_kind + scope + audit counters

wage_line one per employee per run

wage_line_detail per (employee, style, operation) cell

CROSS-CUTTING (core/models.py)

audit_log every hard transition

document / notification shared with Phase 2

24.4 The piece ↔ drawer cycle

`piece.drawer_id` and `drawer.current_piece_id` are the same 1:1 link read from opposite ends — and they are **not interchangeable**:

- The **drawer's** pointer is the **live claim**. A store scan moves state and the leather/lining flags on the row that claims the piece; releasing nulls it when the garment ships.
- The **piece's** pointer is the piece's **own assignment**.

Every card query in barcode/dashboard/production reads through the **claim**; every analytics store query reads through the **assignment**.

The cycle is deliberate. What was missing was a rule for what happens when one end is deleted. Both sides now carry `onDelete="SET NULL"`, which is not a concession — it is already a legal, handled state. A piece with `drawer_id IS NULL` is the "waiting for a drawer" case the pool allocator and the `no_drawer` analytics bucket already speak about. **Deleting a drawer returns its piece to the waiting list rather than deleting the garment's record.**

`use_alter=True` sits on the **drawer** side, and that placement is about DDL ordering, not delete semantics. A cycle means `create_all` cannot topologically sort `{piece, drawer}`; SQLAlchemy silently resolved it by moving **every** foreign key on **both** tables out to `ALTER TABLE`. SQLite cannot execute those, so on the test harness `piece` was created with **no FKs at all** — `sku_id` included. Marking this one back-pointer as the alterable edge lets the sort succeed, `piece` keeps its constraints inline, and the integration test that runs `PRAGMA foreign_keys=ON` can still catch a bad insert order.

24.5 SET NULL on the registry

Every nullable FK on `barcode_registry` carries SET NULL, so a parent row can actually be deleted instead of being pinned forever.

On this table that is the least comfortable rule, because a registry row exists **in order to** name a domain row. It is still right, for two reasons:

- `resolve()` already guards every branch on the FK being present, so a widowed code degrades to "known code, no payload" — it does not crash the scan screen.
- **CASCADE is the alternative, and CASCADE would mean deleting a piece silently destroys its printed label's only record.** That is the exact opposite of "you delete the scannable code, never the person or their record".

The real point: deleting a piece, drawer or employee **row** is not a workflow this system has. Workers are retired, pieces deactivated, drawers recycled. These delete rules exist for administrative cleanup — a mis-imported order, a test batch. If you do use them, sweep the registry for rows whose type-appropriate FK went NULL, or they will keep inflating the per-order minted/balance counts the factory reconciles against.

25. Cross-module choreography

```
ClientService.create_client / add_order
  ▼
ImportService: build_preview → load_preview_into_order      [threadpool, sync loader]
  ▼
BreakdownService.get_table / update_sku / update_style      [DRAFT only]
  ▼
StyleSpecService.confirm                                    ← unlocks release
  ▼
BreakdownService.release_styles                             ★ THE MINT
  ├── premint: Piece rows
  ├── premint: barcode_registry (compact + long alias)
  ├── premint: drawer merge
  └── audit_log
  ▼
AttendanceService.barcode_scan                               ← gates production
  ▼
ProductionService.log_batch                                 ★ THE FLOOR
  ├── Gate 1 ROLE      → 403 whole request
  ├── Gate 2 SKILL     → per-piece warning
  ├── Gate 3 SEQUENCE  → per-piece block
  ├── Gate 4 MERGE     → per-piece block (LINE_STITCHING only)
  ├── EmployeeService: present today?
  ├── MaterialService: decrement stock (cut stages, once per batch)
  └── StyleSpecService: suggested dcm
  ▼
DrawerService.store_scan                                    ← employee → drawer → piece
  ├── infer part from cut history
  ├── completeness → auto RECEIVED
  └── ACCESSORY: issue the kit, spend stock, write the issue ledger
  ▼
DrawerService.send_batch                                    ← unlocks LINE_STITCHING
  ▼
ProductionService.log_batch (PACKAGE_EXPORT)
  └── drawer recycles to WAITING
  ▼
WageService.compute_run                                    ← reads the SAME events
  └── frozen wage_line + wage_line_detail
```

Two properties worth internalising:

1. **Nothing is minted before release.** Upload and commit are safe, reversible operations.
2. **Production events are the single source of truth for both traceability and payroll.** Nothing is counted twice, and the two can never disagree.

26. The key algorithms

26.1 Stage inference

```
def next_chain_stage(completed_codes):
    done = {c.strip().upper() for c in completed_codes}
    chain = ProductionStage.leather_chain()
    furthest = max((i for i, s in enumerate(chain) if s.value in done),
                    default=-1)
    nxt = furthest + 1
    return chain[nxt] if 0 <= nxt < len(chain) else None
```

Pure. No database. Called by both the write path and both read paths.

"Furthest completed, plus one", not "first not completed" — a reworked or out-of-order piece advances from how far it actually got.

26.2 The four gates

GATE 1	ROLE	role in _STAGE_BYPASS_ROLES?	→ pass
		role in STAGE_ROLE_ACCESS[stage]?	→ pass
		operation in operation_access[role]?	→ pass (runtime override)
		else	→ 403, naming the roles that can
GATE 2	SKILL	designation in MULTI_STAGE_DESIGNATIONS?	→ pass (HELPER, SUPERVISOR)
		designation unknown?	→ pass (FAIL-OPEN)
		designation in STAGE_DESIGNATIONS[stage]?	→ pass
		else	→ WARNING, still logged
GATE 3	SEQUENCE	stage.predecessor() is None?	→ pass (cut entries, LINE_STITCHING)
		piece has an event at predecessor?	→ pass
		rework?	→ pass
		else	→ sequence_blocked
GATE 4	MERGE	stage is not LINE_STITCHING?	→ pass
		drawer.state == SENDED?	→ pass
		rework?	→ pass
		else	→ merge_blocked

Gate 2 fails open on purpose. Seeded data carries uncatalogued job titles, and refusing to record work because HR has not backfilled a vocabulary is worse than a warning.

26.3 Completeness

```
needs_lining = EFFECTIVE requirement (style → SKU → cut history)
                NOT the stored piece.needs_lining flag

complete = leather_in AND (lining_in OR NOT needs_lining)

complete → drawer automatically reaches RECEIVED
RECEIVED + a human decision → SENDED
SENDED → the merge gate opens for that piece
```

26.4 Part inference

Reads the piece's cut history and what the drawer is already holding. Returns **LEATHER or LINING and nothing else** — never ACCESSORY, because a mis-inferred kit spends stock. `part_inferred` on the response stays honest about whether the server chose.

26.5 Wage computation

1. GUARD	overlap with an existing run over this window? → 409 naming it
2. SCOPE	piece run → resolve style_code / order_number (REQUIRED) monthly → no scope; any given one is a LABEL
3. POPULATE	piece → ProductionEvent rows in the window, for the scope monthly → salaried employees, priced by the calendar (proration)
4. PRICE	per event: the rate in force ON THAT DAY for (style, operation) unrated → 0, and recorded in unrated_operations
5. FOLD	wage_line_detail per (employee, style, operation) cell wage_line one per employee
6. FREEZE	if freeze=true → CLOSED

Recompute repeats 3–5 with the **same window and scope**, keeping the run id and incrementing `recompute_count`.

26.6 The stock decrement

```
per batch, once, in the same transaction as the events:
  requested = dcm_per_piece × piece_count
  available = on_hand - Σ active reservations
  if requested > available:
    RECORD THE CUT ANYWAY, and return stock_warning
    {lot_id, requested, available_before, short_by, on_hand_after, note}
```

The garment is physically cut. Refusing to record it would lose the traceability the whole system exists for. Silently driving `on_hand` negative would corrupt the ledger. Recording it **and surfacing the shortfall** is the only honest option.

27. Configuration and migration notes

27.1 Settings that change behaviour

Setting	Effect
<code>max_upload_mb</code>	Import cap; streamed, aborts mid-upload
<code>debug</code>	Enables <code>create_all</code> at startup — development only
Shift policy	Not config — it is a database row, editable by HR
Drawer pool size	Not config — a real table, grown by DM/MD

27.2 The native-enum trap

`app_user.role` is a native Postgres enum. Adding a member to `UserRole` in Python is **not enough** — Postgres needs the label added to the type:

```
if op.get_bind().dialect.name == "postgresql":
    op.execute("ALTER TYPE user_role ADD VALUE IF NOT EXISTS 'LINING_MANAGER'")
```

And it must be the member NAME, uppercase — not the value.

`Enum(UserRole, ...)` persists the enum **member name**, because this project never passes `values_callable`. For `SECURITY = "security"` the driver sends `'SECURITY'`. Adding the lowercase `'security'` creates a label the ORM will never emit, and the role stays unusable — the INSERT still dies with

`invalid input value for enum user_role: "SECURITY" even though 'security' is on the type.`

This bit `lining_manager`, `security` and `merchandise`. Write the name exactly as it appears left of the `=` in `core/enums.py`. The schema's other native enums (`attendance_source`, `wage_type`, `run_status`) are all uppercase names — match them.

27.3 Model imports

Every model module must be imported in `main.py` so `Base.metadata` sees every table. A missed import makes Alembic autogenerate try to **DROP** the table.

`barcode.models` holds the barcode, material, drawer, supplier, style-spec and issue-ledger tables. `dashboard` and `analytics` own none — verify that before adding a model-import line for them.

27.4 Router locking

Manager-only routers are wrapped with `block_employees`. **Auth, users, attendance and `/barcode/resolve` stay open** at the router level — the attendance write routes do their own operator gate, and resolve must stay reachable from the attendance screen.

27.5 Route ordering

FastAPI matches in **registration order**. Three files depend on it:

File	The trap
<code>clients/router.py</code>	<code>GET /clients/styles</code> must precede <code>GET /clients/{client_id}</code> — "styles" is not a UUID, so the request would 422 instead of reaching the handler
<code>drawers/router.py</code>	<code>/pool</code> , <code>/by-code/{code}</code> and <code>/send</code> must precede <code>/ {drawer_id}</code>
<code>wages/router.py</code>	Every static segment must precede <code>/runs/{run_id}</code>

Style codes are **query** parameters, not path segments, for a related reason: a code like `JP-CLERMONT_VEST` is fine in a path, but the day someone imports a style whose name slugs to something containing a slash, a path parameter breaks and a query parameter does not.

27.6 Known traps — do not reintroduce

Trap	Symptom
Importing <code>api</code> from <code>sqlalchemy.event</code>	Wrong object; routers never register. Use <code>app.include_router</code>
Defining <code>lifespan</code> twice	The second silently overrides the first
Hoisting a lazy cross-module import	Import cycle. <code>production.service → materials.service</code> , <code>→ drawers.service</code> and <code>drawers.service → production.models</code> are all inside the method that uses them
Adding an enum member without the PG migration	The role is unusable
Adding the enum value instead of the name	The role is <i>still</i> unusable, confusingly
Encoding stage order against <code>Operation.sequence</code>	A stray UPDATE reorders the factory
Duplicating <code>next_chain_stage</code>	Read and write paths drift
Reading <code>piece.needs_lining</code> directly	It is written once at release and wrong for most of a live order. Use the effective resolver

28. A code-review checklist

Layering

- [] Router contains no business logic — dependency, **barcode→id resolution**, delegate.
- [] Commits live in the service; the whole scan is one transaction.
- [] All DB access is in the repository.
- [] Cross-module calls go **service → service**, and the import is **lazy**.

The production contract

- [] **No endpoint accepts a stage from the client.** Stage is derived.
- [] Screen context is derived from the role; only DM/MD/HR may override.
- [] Both doors converge on ids before the service sees anything.
- [] Gate 1 is whole-request; gates 2–4 are per-piece.
- [] Every per-piece rejection lands in `blocked[]` with `{piece, stage, gate, reason}`.
- [] `next_chain_stage` is called, never reimplemented.

Correctness

- [] Blocking work runs in `run_in_threadpool`.
- [] Money and quantities are `Decimal`.
- [] `available` is derived, never stored.
- [] Stock decrements **once per batch**, in the same transaction as the events.
- [] The lot link is on the **event**, never on the piece.
- [] Idempotent writes are idempotent — check-in re-tap, accessory re-scan, close-run.

State and safety

- [] Hard transitions write an `audit_log` row: release, receive, send, barcode reissue/deactivate, material receipt, kit issue, spec confirm, run reopen.
- [] Nothing is hard-deleted where a soft-deactivate or retire would do.
- [] A retired barcode returns **410**, not 404.
- [] A removed endpoint returns **410 naming the replacement**, not 404.
- [] **Partial accept**: batch operations report per-item outcomes and commit the good ones.
- [] **When unsure, ask** — ambiguous lots, undeclared lining, unconfirmed specs all stop and surface the choice.

Contract

- [] New status columns are `str`-Enum in VARCHAR — **not** a native PG enum.
 - [] A new `UserRole` member ships with an `ALTER TYPE ... ADD VALUE 'NAME'` migration.
 - [] New models are imported in `main.py`.
 - [] New endpoints carry a role gate, or are deliberately and documentedly open.
 - [] Tenancy: any client-reachable read threads `client_scope`, and a cross-tenant id yields **404**.
 - [] Cost and pay data is gated to HR/DM/MD — never bare `get_current_user`.
-

APPENDICES

Appendix A — Glossary

Term	Meaning
Piece	ONE physical garment. The tracked unit. Identity is <code>(sku_id, seq)</code>
SKU	A colour + size combination of a style, with a quantity ordered
Breakdown sheet	The workbook that turns a client order into styles and SKUs
Release	The hard, audited transition that mints pieces, barcodes and drawer merges
Mint	To create a barcode. Irreversible
Compact code	The short <code>PC-004821</code> that is printed and scanned
Alias	The long legacy code kept scannable after the compact switch
dcm	Square decimetres — the unit leather is measured and consumed in
Lot	A batch of material with its own barcode and its own three stock numbers
Drawer	A physical storage slot. Static code, recycling state
Merge	Assigning a piece to a drawer at release
Merge gate	The completeness rule blocking LINE_STITCHING until the drawer is SENDED
Completeness	Leather present, and lining present if the garment needs one
Kit	A garment's accessory set, issued from the style's recipe
Recipe	<code>style_material_spec</code> — what one garment of this style needs
Rework	Logging a piece at a stage it already passed. Permitted, flagged
Screen context	LEATHER_CUT / LINING_CUT / PIPELINE — derived from the role
Designation	An employee's skill. Uppercase, controlled vocabulary
Operator	A login permitted to write attendance: Security, HR, MD, DM
Proxy	An attendance entry made by an operator for a worker
Piece run	A payroll for PIECE_RATE workers. Requires a style or order scope
Monthly run	A payroll for salaried staff, priced by the calendar
Frozen run	A CLOSED wage run — the document the cash was counted against
Sea cut-off	The last day to make sea freight. Missing it means air
Bottleneck	The deepest queue — the stage with the most work waiting

Appendix B — Complete enum reference

`UserRole` `managing_director` · `direct_manager` · `cutting_manager` · `lining_manager` · `stitching_manager` · `store_manager` · `supervisor` · `hr` · `security` · `merchandiser` · `client` · `viewer` · `employee` (*legacy*)

`ATTENDANCE_OPERATOR_ROLES` `security` · `hr` · `managing_director` · `direct_manager`

`ProductionStage` `LEATHER_CUTTING` · `LINING_CUTTING` · `FUSING` · `PASTING` · `LINE_STITCHING` · `SHELL_STITCHING` · `FINAL_FINISH` · `FINAL_INSPECTION` · `PACKAGE_EXPORT`

ScreenContext LEATHER_CUT · LINING_CUT · PIPELINE

Designation CUTTER · LINING_CUTTER · FUSER · PASTER · LINE_TAILOR · SHELL_TAILOR · FINISHER · INSPECTOR · PACKER · TAILOR · HELPER · SUPERVISOR · TRIMMER · CHEMICAL_TECHNICIAN · SECURITY · MERCHANDISER · STITCHING_INSTRUCTOR · QC_INSPECTOR *Multi-stage (no skill gate):* HELPER · SUPERVISOR

BarcodeType PIECE · LEATHER_LOT · LINING_LOT · ACCESSORY_LOT · EMPLOYEE · DRAWER

BarcodeStatus active · retired

DrawerState waiting · merged · holding_leather · holding_lining · holding_both · received · sented

DrawerPart LEATHER · LINING · ACCESSORY *(never inferred)*

MaterialCategory LEATHER · LINING · ACCESSORY

MaterialSubtype RIBS · KNIT · PLAIN_LINING · BUTTON · ZIP · THREAD · OTHER

KitStatus NOT_REQUIRED · PENDING · PARTIAL · ISSUED

MaterialIssueSource STORE_KIT · CUT *(reserved)* · MANUAL

ReceiptStatus approved · rejected

SupplierOrderStatus ordered · arrived

ProductionReleaseStatus DRAFT · RELEASED · CANCELLED

WageType monthly · piece_rate

WageRunKind piece · monthly · ~~combined~~ *(legacy)*

RunStatus open · closed

AttendanceSource self · proxy

ShipMode sea · air

Appendix C — Audit log actions

Action	Written when
DRAWER_RECEIVED	A drawer reaches completeness
DRAWER_SENDED	A batch is released to line-stitching
EMPLOYEE_BARCODE_REISSUE	A card is replaced
EMPLOYEE_BARCODE_DEACTIVATE	A worker leaves
MATERIAL_RECEIVED	A delivery is received
MATERIAL_KIT_ISSUED	An accessory kit is issued — it spends stock
MATERIAL_ISSUED_MANUAL	An off-spec correction
STYLE_MATERIAL_SPEC_CONFIRMED	A recipe is signed off
STYLE_MATERIAL_SPEC_AMENDED	A recipe is changed

Plus release, style cancel, lot adjust, lot retire, and every wage-run reopen/recompute/delete.